CODEATLAS AI: INTELLIGENT SOURCE CODE ANALYSIS AND AUTOMATED DOCUMENTATION GENERATION USING NLP AND TRANSFORMER MODELS

A project report submitted by

ASHER DUTHIE ([Register No.])
JENNIFER GRACE JOSEPH ([Register No.])

in partial fulfillment of the requirements for the course

26CS301 – NATURAL LANGUAGE PROCESSING

of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

under the supervision of

[Supervisor Name], [Designation]

DIVISION OF COMPUTER SCIENCE AND ENGINEERING
KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed-to-be University under Sec. 3 of the UGC Act, 1956)
Karunya Nagar, Coimbatore – 641 114, India

NOVEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"CodeAtlas AI: Intelligent Source Code Analysis and Automated Documentation Generation Using NLP and Transformer Models"** is a bonafide record of the work done by **ASHER DUTHIE** ([Register No.]) and **JENNIFER GRACE JOSEPH** ([Register No.]) during the academic year 2026–2027, in partial fulfillment of the requirements of the course 26CS301 – Natural Language Processing for the degree of Bachelor of Technology in Computer Science and Engineering of Karunya Institute of Technology and Sciences, and that the work was carried out under my supervision.

SIGNATURE	SIGNATURE
Dr. G. Jaspher W. Kathrine	**[Supervisor Name]**
Head of the Division	[Designation], Supervisor
Division of Computer Science and Engineering	Division of Computer Science and Engineering

Submitted for the Project Viva Voce held on _____

Internal Examiner	**External Examiner**

PROJECT COMPLETION CERTIFICATE

[Insert the signed project / course completion certificate issued by the Division here.]

ACKNOWLEDGEMENT

First and foremost, we praise and thank Almighty God for His grace and blessings throughout this work.

We express our gratitude to our Founder, **Dr. D. G. S. Dhinakaran**, our beloved Chancellor, **Dr. Paul Dhinakaran**, and our Vice Chancellor, **Dr. R. Elijah Blessing**, for the opportunity to carry out this project. We thank our Registrar, **Dr. S. J. Vijay**, for his encouragement and support.

We are grateful to our Dean, **Dr. D. Nirmal**, and to the Head of the Division of Computer Science and Engineering, **Dr. G. Jaspher W. Kathrine**, for their guidance and for the facilities provided.

We thank our project supervisor, **[Supervisor Name]**, [Designation], for valuable advice, review and constant support at every phase of this work.

We also thank all the faculty members and staff of the Division, our classmates, and our families for their encouragement throughout the course of this project.

ABSTRACT

Software documentation is often missing or out of date. Neural and large-language-model documentation tools can describe undocumented code fluently, but they do not show which source lines justify each statement, so their errors are hard to detect. **CodeAtlas AI** is a platform that analyses Python and Java repositories as both programs and natural-language corpora. Its aim is that every statement about the code should be traceable to source evidence and open to human verification.

The project is organised into five phases. Phase I ingests files, ZIP archives and public GitHub repositories with bounded and hostile-input-safe validation, preserves original bytes in a SHA-256 content-addressed store, and preprocesses code into three aligned views (original, normalized and NLP) using Tree-sitter tokens, comments, JavaDoc and Python docstrings, with identifier splitting. Phase II extracts symbols of eleven kinds and five kinds of relationships, resolves every relationship conservatively as confirmed, ambiguous, unresolved or external, finds import cycles with Tarjan's algorithm, and computes deterministic corpus statistics such as vocabulary size, type–token ratio, documentation-term frequencies, naming patterns and documentation coverage. Phase III delivers a React and TypeScript dashboard: a shared 3D/2D visualization system with provenance on every chart, an Overview page, a force-directed Knowledge Graph, a Code Explorer and a claim Verification workspace. The dashboard is backed by separate Demo and Live data layers that are never silently mixed.

The FastAPI backend passed 80 tests with 94% coverage, and the frontend passed 113 unit tests together with its browser suites. On an Intel HD 520 integrated GPU the 3D Knowledge Graph renders 2,000 nodes at 60 frames per second. Extraction accuracy is not yet reported, because independent gold annotations are pending (Phase II is REVIEW_PENDING). Transformer-based documentation generation with CodeT5 and automatic claim verification are planned for Phases IV and V and are not yet implemented.

Keywords: natural language processing, source code analysis, automated documentation, traceability, Tree-sitter, lexical diversity, knowledge graph, claim verification, CodeT5

TABLE OF CONTENTS

ACKNOWLEDGEMENT i
ABSTRACT ii
LIST OF FIGURES v
LIST OF TABLES vii
LIST OF SYMBOLS AND ABBREVIATIONS ix
CHAPTER 1 INTRODUCTION 1
1.1 OBJECTIVE 1
1.2 PROBLEM STATEMENT 3
1.3 CHAPTER-WISE SUMMARY 3
CHAPTER 2 LITERATURE SURVEY 5
2.1 EXISTING SYSTEM 5
2.1.1 Overview of the Existing System 5
2.1.2 Purpose and Goals of the Existing System 7
2.2 SYSTEM ARCHITECTURE AND COMPONENTS 7
2.3 DATA MANAGEMENT 8
2.4 SYSTEM PERFORMANCE 9
2.5 LIMITATIONS AND CHALLENGES 9
CHAPTER 3 PROPOSED SYSTEM 11
3.1 DETAILED DESIGN 11
3.2 METHODOLOGY 15
3.2.1 Collection and Source Preservation (Phase I) 15
3.2.2 Lexical Preprocessing into Three Views 16
3.2.3 Structure Extraction (Phase II) 17
3.2.4 Conservative Relationship Resolution 17
3.2.5 Exploratory Text Statistics 18
3.2.6 Evaluation Framework 19
3.2.7 Visualization and Interaction Design 19
3.2.8 Claim Review Workflow 20
3.3 MODULES 22
3.4 DATABASE DESIGN 26
3.4.1 ER Diagram 27
3.4.2 Tables or Entities 27
CHAPTER 4 IMPLEMENTATION 29
4.1 MODULE IMPLEMENTATION 29
4.1.1 Backend Service and API 29

4.1.2 Preprocessing and Analysis 30
4.1.3 Frontend Shell and Data Layer 32
4.1.4 Visualization System (M1) 32
4.1.5 Overview and Knowledge Graph (M3, M4) 33
4.1.6 Code Explorer and Verification Workspace 34
4.2 TESTING 36
4.2.1 Backend Testing (Observed) 36
4.2.2 Frontend Testing (Observed) 37
4.2.3 Performance Testing (Observed) 38
4.2.4 Extraction Accuracy (Pending) 39
4.2.5 Suggested Test Procedures (Not Executed) 39
4.2.6 Known Limitations 39
CHAPTER 5 CONCLUSION AND FUTURE SCOPE 41
5.1 CONCLUSION 41
5.2 FUTURE SCOPE 42
APPENDIX A: SOURCE CODE 43
APPENDIX B: SCREENSHOTS 46
REFERENCES 53

LIST OF FIGURES

Fig 2.1 Pipeline of a Conventional Comment-Driven Documentation Generator.....5
Fig 2.2 Generic Pipeline of Neural Code Summarization Systems.....6
Fig 3.1 System Architecture of CodeAtlas AI.....12
Fig 3.2 Use Case Diagram.....13
Fig 3.3 Data Flow Diagram, Level 0 (Context Diagram).....13
Fig 3.4 Data Flow Diagram, Level 1.....14
Fig 3.5 Frontend Data Layer with Demo and Live Adapters.....14
Fig 3.6 Shared 3D/2D Visualization Pipeline.....15
Fig 3.7 Phased Methodology and Documented Status.....15
Fig 3.8 Flowchart of Ingestion and Preprocessing.....16
Fig 3.9 Flowchart of Relationship Resolution.....18
Fig 3.10 Claim Review Workflow.....20
Fig 3.11 Sequence Diagram: Import, Preprocessing and Analysis.....21
Fig 3.12 Sequence Diagram: Dashboard Page in Live Mode.....22
Fig 3.13 Entity–Relationship Diagram.....27
Fig 4.1 Token Distribution by Type on the Evaluation Corpus (Measured).....31
Fig 4.2 Identifier Frequencies Generated by the Phase II Report Script (Measured).....31
Fig 4.3 Documentation Coverage Chart Generated by the Phase II Report Script (Measured).....32
Fig 4.4 Overview Page in 2D Mode (Demo Data, Illustrative).....33
Fig 4.5 Knowledge Graph in 3D with a Selected Node (Demo Data, Illustrative).....34
Fig 4.6 Code Explorer with Hover Card and Inspector (Demo Data, Illustrative).....35
Fig 4.7 Verification Workspace with Evidence Lines (Demo Data, Illustrative).....36
Fig 4.8 Passing Frontend Unit Tests by Milestone (Recorded).....38
Fig B.1 Design-system showcase: grouped and stacked bars, 3D.....46
Fig B.2 Design-system showcase: grouped and stacked bars, 2D.....47
Fig B.3 Measured word clouds of identifiers and documentation terms from the Phase II analysis, 3D.....47
Fig B.4 Command palette in the dashboard shell.....48
Fig B.5 Settings page with data mode and Live connection.....48
Fig B.6 Overview page in 3D mode.....49
Fig B.7 Overview page at phone width (390 px).....50
Fig B.8 Knowledge Graph in 2D with a selected node.....51
Fig B.9 Knowledge Graph in the light theme.....51
Fig B.10 Code Explorer, Relationships tab of the inspector.....52

Fig B.11 Verification workspace filtered to unsupported claims 52

LIST OF TABLES

Table 1.1 Objectives Mapped to Project Phases and Current Status 2
Table 2.1 Summary of Literature Reviewed 6
Table 2.2 Feature Comparison of Existing Approaches and CodeAtlas AI 8
Table 2.3 Limitations of Existing Systems and the CodeAtlas AI Response 10
Table 3.1 Technology Stack 12
Table 3.2 Exploratory Statistics and Their Computation 18
Table 3.3 Force-Directed Layout Parameters (Knowledge Graph) 20
Table 3.4 Module 1: Ingestion and Validation 22
Table 3.5 Module 2: GitHub Acquisition 22
Table 3.6 Module 3: Content-Addressed Storage 23
Table 3.7 Module 4: Lexical Preprocessing 23
Table 3.8 Module 5: Structure Extraction 23
Table 3.9 Module 6: Exploration and Statistics 24
Table 3.10 Module 7: Extraction Evaluation 24
Table 3.11 Module 8: API, Jobs and Orchestration 24
Table 3.12 Module 9: Frontend Data Layer 24
Table 3.13 Module 10: Shared Visualization System 25
Table 3.14 Module 11: Overview Page 25
Table 3.15 Module 12: Knowledge Graph 25
Table 3.16 Module 13: Code Explorer 25
Table 3.17 Module 14: Verification Workspace 26
Table 3.18 Module 15: Pending Modules 26
Table 3.19 Structure of the projects Table 27
Table 3.20 Structure of the snapshots Table 27
Table 3.21 Structure of the source_files Table 28
Table 3.22 Structure of the jobs Table 28
Table 3.23 Structure of the analysis_symbols Table 28
Table 3.24 Structure of the analysis_relationships Table 28
Table 4.1 REST API Operations (all under /api/v1) 29
Table 4.2 Measured Results on the Evaluation Corpus (Phase II) 30
Table 4.3 Backend Test Files 36
Table 4.4 Recorded Backend Test Results 37
Table 4.5 Frontend Unit Test Files (Vitest, 113 Passing at M4) 37
Table 4.6 Recorded Browser Test Results (Playwright, Chromium) 38

Table 4.7 Knowledge Graph Frame Rate on Intel HD 520 (Recorded, Headless Chrome) 38
Table 4.8 Planned Test Procedures 39
Table 5.1 Final Status Summary 41

LIST OF SYMBOLS AND ABBREVIATIONS

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
AST	Abstract Syntax Tree
BLEU	Bilingual Evaluation Understudy
CAIR	Phase I lexical intermediate representation (data contract, version 1.0)
CI	Continuous Integration
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ER	Entity–Relationship
ETag	HTTP entity tag
fps	Frames per second
GL	Graphics Library (software GL: CPU-rendered WebGL)
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LLM	Large Language Model
LOD	Level of Detail
LSI	Latent Semantic Indexing
NER	Named Entity Recognition
NLP	Natural Language Processing
ORM	Object–Relational Mapping
p50, p95	50th and 95th percentile
PEP	Python Enhancement Proposal
PNG	Portable Network Graphics
REST	Representational State Transfer
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
SCC	Strongly Connected Component
SHA-256	Secure Hash Algorithm, 256-bit
SNI	Server Name Indication
SVG	Scalable Vector Graphics
TTR	Type–Token Ratio
UI	User Interface
URL	Uniform Resource Locator
WAI-ARIA	Web Accessibility Initiative – Accessible Rich Internet Applications
WebGL	Web Graphics Library

List of Symbols

Symbol	Meaning
N	Number of non-comment tokens in a snapshot
V, |V|	Set of distinct token spellings; vocabulary size
S_d	Set of documentable symbols
D	Set of documented symbols, D ⊆ S_d
G = (M, E)	Confirmed import graph with modules M and edges E
deg(v)	Degree of graph node v
r(v)	Drawn radius of node v
z	Zoom factor of the graph view
P, G (evaluation)	Predicted items and gold-standard items
α (alpha)	Simulation temperature of the force layout

CHAPTER 1
INTRODUCTION

Software is written once but read many times. Developers who join a project, review a change or fix a defect spend much of their time reconstructing what a module does, which other modules depend on it and whether its comments are still true. Empirical studies of practice report that documentation is frequently missing, incomplete or out of date, and that developers distrust it for exactly that reason [36], [37]. Source code is, however, also a form of text. It contains natural-language identifiers, comments, docstrings and JavaDoc blocks, and research on the "naturalness" of software has shown that code is repetitive and predictable in ways that statistical language models can exploit [20], [21]. This makes software repositories a natural subject for Natural Language Processing (NLP).

CodeAtlas AI is the project reported here. Its full title is *Intelligent Source Code Analysis and Automated Documentation Generation Using NLP and Transformer Models*. The intended product lets a user upload a Python or Java project, or import a public GitHub repository, and then receive a searchable code map, multi-level explanations, a knowledge graph of the code, documentation analytics and generated documentation. The feature that distinguishes it from a plain documentation generator is *traceability*: every statement the system makes about the code is meant to be linked back to the exact source lines that support it, and to be marked Verified, Uncertain or Unsupported so that a human reviewer can check it.

The project is organised into five phases. Phase I (collection and preprocessing) and Phase II (exploratory text analysis and structure extraction) form the backend foundation. Phase III builds the NLP visualization layer and the web dashboard. Phase IV (NLP model development) and Phase V (advanced implementation, including documentation generation and export) are planned but have not started in the repository. This report describes all five phases, and it states the verified status of each one plainly: what has been implemented and tested, what is partial, and what remains future work.

1.1 OBJECTIVE

The overall objective of CodeAtlas AI is to build a platform that analyses a source code repository as both a program and a natural-language corpus, and that presents the results so that

every claim about the code can be traced to evidence. The specific objectives are listed below and are mapped to project phases in Table 1.1.

1. To collect Python and Java source code safely from file uploads, ZIP archives and public GitHub repositories, preserving the original bytes exactly and identifying every snapshot by a SHA-256 content hash.
2. To preprocess source code into three aligned text views (the exact original, a newline-normalized view and an NLP view) with lexical tokens, comments, docstrings and JavaDoc extracted using Tree-sitter grammars and the Python `ast` module.
3. To extract program structure (modules, classes, functions, methods, constructors and fields) and typed relationships (contains, imports, inherits, implements and calls), and to resolve those relationships conservatively so that uncertain links are labelled rather than guessed.
4. To compute reproducible exploratory statistics over the code-as-text corpus, including token and identifier distributions, vocabulary size, lexical diversity, documentation-term frequencies, naming patterns and documentation coverage.
5. To visualize these results in an interactive dashboard with a shared 3D/2D chart system, a force-directed knowledge graph, a code explorer and a claim verification workspace, with every chart labelled by the provenance of its data.
6. To prepare, for later phases, transformer-based documentation generation (for example CodeT5 [17]) and claim verification whose output is grounded in source evidence.

Table 1.1 Objectives Mapped to Project Phases and Current Status

No.	Objective	Phase	Status
1	Safe collection and byte-exact source preservation	I	PASS
2	Lexical preprocessing into three text views	I	PASS
3	Structure extraction and conservative edge resolution	II	PARTIAL / REVIEW_PENDING
4	Exploratory text statistics	II	PARTIAL / REVIEW_PENDING
5	Interactive NLP visualization and dashboard	III	IN PROGRESS (M1–M4 complete)
6	Transformer documentation generation and claim verification	IV, V	NOT STARTED

1.2 PROBLEM STATEMENT

Documentation tools in common use, such as Javadoc, Doxygen and Sphinx [1]–[3], only reformat comments that developers have already written. They cannot describe undocumented code, and they cannot detect when a comment has drifted away from the code it describes. Neural code summarization models and large language models (LLMs) can produce fluent descriptions of undocumented code [6], [14], [17], [18], but they do so without showing which lines justify each sentence. Generated text can therefore contain plausible but false statements, a failure known as hallucination [26], and a reader has no inexpensive way to tell a true sentence from a false one.

Before any generation is attempted, a further problem must be solved. Repository content arrives from untrusted sources (archives can contain path-traversal entries, symbolic links or decompression bombs, and remote downloads can be redirected), and any analysis is only as reliable as the text and structure it is computed from. The problem addressed by this project is therefore stated as follows:

To design and build a system that safely ingests Python and Java repositories, transforms them into reproducible NLP-ready representations, extracts their structure with explicit confidence in every relationship, and presents the results so that each statement about the code is traceable to source evidence and open to human verification. Model-generated documentation is to be added only on top of this verified foundation.

1.3 CHAPTER-WISE SUMMARY

Chapter 1, Introduction, introduces the motivation for analysing source code as natural-language text, states the objectives and the problem, and summarises the report.

Chapter 2, Literature Survey, reviews existing documentation generators, neural code summarization and pre-trained code models. It covers their architectures, datasets, evaluation practice and limitations, and identifies the gap in traceability and verification that CodeAtlas AI targets.

Chapter 3, Proposed System, presents the architecture, use cases, data flow diagrams, the phased methodology (ingestion, preprocessing, structure extraction, edge resolution, exploratory statistics, evaluation framework and visualization), a description of every module, and the database design with its ER diagram.

Chapter 4, Implementation, describes how each module was implemented, presents measured results on the evaluation corpus and the Phase III dashboard pages, and reports the testing carried out. Recorded test results are kept separate from procedures that have not yet been executed.

Chapter 5, Conclusion and Future Scope, summarises what has been achieved and what remains, and outlines Phase IV and Phase V.

Appendix A contains selected source code and **Appendix B** contains screenshots of the working system.

CHAPTER 2
LITERATURE SURVEY

This chapter surveys the systems and research that CodeAtlas AI builds on. The survey is organised by the headings of the prescribed format: an overview of existing systems and their goals, their architecture and components, how they manage data, how their performance is measured, and their limitations. Table 2.1 summarises the principal works reviewed.

2.1 EXISTING SYSTEM

2.1.1 Overview of the Existing System

Existing approaches to code documentation fall into three families.

Comment-driven documentation generators. Javadoc [1], Doxygen [2] and Sphinx autodoc [3] parse source files, collect structured documentation comments (`@param`, `@return`, `:param:` and similar markup) and render them into browsable HTML or PDF reference manuals. They are deterministic and widely trusted, but they only present what developers have already written (Fig 2.1).

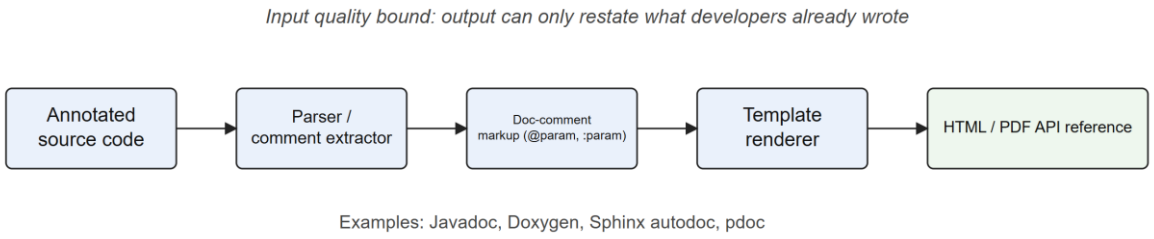


Fig 2.1 Pipeline of a Conventional Comment-Driven Documentation Generator

Template and information-retrieval summarization. Early automatic summarization of code selected salient terms or statements. Sridhara *et al.* generated summary comments for Java methods from heuristics over identifiers and statements [4], and Haiduc *et al.* applied text-retrieval techniques such as latent semantic indexing to produce term-based summaries [5]. These methods made the first link between NLP and code documentation, but their output was limited to templates and keywords.

Neural and pre-trained summarization. CODE-NN [6] introduced an attention-based neural model that generates natural-language summaries of C# and SQL snippets. DeepCom [7] used

abstract syntax tree (AST) traversals for Java comment generation, and code2vec and code2seq [8], [9] represented code as sets of AST paths. LeClair *et al.* combined token and AST encoders [10]. The Transformer architecture [13] was applied to summarization by Ahmad *et al.* [14]. Pre-trained models then followed: CodeBERT [16], adapted from BERT [15], and CodeT5 [17], an identifier-aware encoder–decoder. Large models trained on code, such as Codex [18], now generate documentation and code inside development tools. Fig 2.2 shows the generic pipeline.

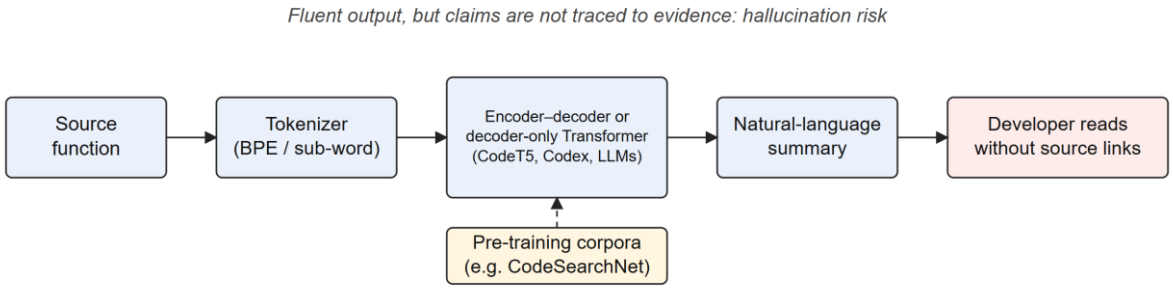


Fig 2.2 Generic Pipeline of Neural Code Summarization Systems

Table 2.1 Summary of Literature Reviewed

Ref.	Work (year)	Technique	Contribution	Limitation relevant here
[1]–[3]	Javadoc, Doxygen, Sphinx	Doc-comment parsing and templates	Reliable API reference from existing comments	Cannot document undocumented code or detect stale comments
[4]	Sridhara et al. (2010)	Heuristic sentence generation	First automatic summary comments for Java methods	Template output; heuristic rules per construct
[5]	Haiduc et al. (2010)	Text retrieval (VSM, LSI)	Term-based code summaries	Keywords rather than sentences
[6]	Iyer et al. (2016)	Attention LSTM	CODE-NN neural summaries	Small corpora; no evidence linking
[7]	Hu et al. (2018)	AST-traversal seq2seq	DeepCom for Java	Generated text unverified
[8], [9]	Alon et al. (2019)	AST path embeddings	code2vec and code2seq	Method-level only
[10]	LeClair et al. (2019)	Token + AST encoders	Better summaries of subroutines	Dataset-dependent quality
[12]	Husain et al. (2019)	Benchmark corpus	CodeSearchNet, six languages	Docstring-derived labels are noisy
[14]	Ahmad et al. (2020)	Transformer, relative positions	Transformer code summarization	No claim grounding
[16]	Feng et al. (2020)	Bimodal pre-training	CodeBERT	Encoder only; needs a decoder for generation
[17]	Wang et al. (2021)	Identifier-aware T5	CodeT5 for understanding and generation	Outputs fluent but unverifiable text
[18]	Chen et al. (2021)	Large code language model	Codex, HumanEval	Hallucination; closed weights

Ref.	Work (year)	Technique	Contribution	Limitation relevant here
[23]	Roy et al. (2021)	Human study of metrics	BLEU-style scores track human judgement poorly	Shows that metric gains need care
[26]	Ji et al. (2023)	Survey	Taxonomy of hallucination in NLG	Mitigation remains open

2.1.2 Purpose and Goals of the Existing System

Comment-driven generators aim to give a consistent, navigable reference for a public API and to keep the documentation next to the code so that the two change together. Summarization research aims to reduce the documentation burden by producing comments for code that has none, and to help developers comprehend unfamiliar code faster. Pre-trained code models broaden the goal to a family of tasks (summarization, search, translation, generation) learned from a single large corpus [16], [17], [19]. Code intelligence tools built on parsers such as Tree-sitter [29] aim to make structure (definitions, references, call sites) navigable at the scale of an entire repository.

None of these goals includes showing *why* a generated statement is true. That is the gap this project addresses.

2.2 SYSTEM ARCHITECTURE AND COMPONENTS

Despite their differences, existing systems share four architectural components.

- **Parsing front end.** Documentation generators use language-specific parsers, while research systems use either a token stream or an AST. Tree-sitter provides incremental, error-tolerant concrete syntax trees for many languages and is used by modern editors and code intelligence tools [29]. Python additionally exposes its own parser through the `ast` module [45].
- **Representation.** Representations range from flat sub-word token sequences [13], [17] through AST paths [8] to program graphs that add data-flow and control-flow edges [44].
- **Generator.** Template engines are used in deterministic tools. Neural systems use an encoder–decoder (LSTM or Transformer), or a decoder-only language model [18].
- **Presentation.** Output is usually static HTML or an inline editor hint. Visual exploration of repository structure is normally a separate tool, typically a node-link diagram drawn with a force-directed layout [32], [33].

Table 2.2 compares representative existing systems with the target capabilities of CodeAtlas AI. The CodeAtlas column distinguishes features that are implemented from features that are planned.

Table 2.2 Feature Comparison of Existing Approaches and CodeAtlas AI

Capability	Doc generators [1]–[3]	Neural summarizers [6]–[17]	LLM assistants [18]	CodeAtlas AI
Works on undocumented code	No	Yes	Yes	Planned (Phase IV)
Byte-exact source snapshot	No	No	No	Implemented
Structure graph with confidence per edge	No	No	No	Implemented (4 resolution states)
Code-as-text statistics (tokens, lexical diversity, coverage)	No	Research only	No	Implemented
Claim linked to evidence lines	No	No	Partial (citations)	Review UI built; verifier planned
Interactive 3D/2D visual analytics	No	No	No	Implemented (M1–M4)

2.3 DATA MANAGEMENT

Neural summarization depends on large parallel corpora of code paired with natural-language descriptions. CodeSearchNet pairs functions in six languages with the first paragraph of their docstrings [12], and CodeXGLUE gathers that corpus with other tasks into a common benchmark [19]. LeClair and McMillan showed that the way such corpora are split matters greatly. Splitting by function rather than by project lets near-identical code from the same project appear in both training and test sets, which inflates reported scores [11]. Allamanis measured the same effect for code duplication in general [22]. CodeAtlas AI follows this lesson by detecting duplicate files by content hash during ingestion and reporting them explicitly (Chapter 4).

Preprocessing choices also matter. Code tokens are usually split into sub-tokens (camelCase and snake_case boundaries), because identifiers carry most of the natural-language signal in code [20], [21]. Comment markers must be removed without losing the comment text. Unlike ordinary NLP pipelines, stop-word removal is risky for documentation text, because words such as *not*, *only* and *before* carry precise meaning about behaviour. CodeAtlas AI therefore keeps stop words in its NLP view.

Deterministic tools store nothing beyond the generated pages. Research systems typically store preprocessed tensors. Neither keeps an immutable link between an output and the exact bytes it was derived from, which is the foundation that claim verification needs.

2.4 SYSTEM PERFORMANCE

Code summarization is usually evaluated with n-gram overlap metrics borrowed from machine translation and text summarization: BLEU [24], ROUGE-L [25] and METEOR. These metrics reward lexical overlap with a single reference comment. Roy *et al.* [23] carried out a human study and found that small differences in such metric scores do not reliably indicate that one summary is better than another. Ji *et al.* [26] note that overlap metrics do not measure factual consistency at all. Factuality-oriented methods break generated text into atomic claims and check each one against a knowledge source [28], or ground generation in retrieved evidence [27]. These ideas carry over directly to code, where the source itself is the evidence.

For structure extraction, the appropriate measures are precision, recall and F1 over symbols and relationships, compared against a human-annotated gold set. Performance in the systems sense (latency, throughput, frame rate of interactive views) is rarely reported in the summarization literature. It matters for a dashboard that renders hundreds of graph nodes, and it is measured for CodeAtlas AI in Chapter 4.

2.5 LIMITATIONS AND CHALLENGES

The review identifies the following limitations of existing systems, which together motivate CodeAtlas AI. Table 2.3 maps each limitation to the project's response and to the status of that response.

- **No traceability.** Generated documentation is not linked to the lines that justify it, so readers must either trust it or re-read the code.
- **Hallucination.** Fluent models can state behaviour the code does not have (for example, that a function is idempotent or raises a specific error) [26].
- **Unsafe or irreproducible inputs.** Research pipelines assume clean corpora. A deployed tool must handle hostile archives and redirecting downloads, and must be able to reproduce a result from the same bytes.
- **Over-confident structure.** Static extraction from dynamic languages is inherently uncertain, and tools rarely say how certain a call or import link is.

- **Weak evaluation.** Overlap metrics correlate poorly with human judgement [23], and gold annotations for structure extraction are expensive to produce.
- **Separate visualization.** Statistics, graphs and documentation usually live in different tools with different data, which makes cross-checking difficult.

Table 2.3 Limitations of Existing Systems and the CodeAtlas AI Response

Limitation	CodeAtlas AI response	Status
No traceability	Every symbol and edge carries an evidence span (file, line, column); the Explorer and Verification pages show source chips and highlighted lines	Implemented for structure; claims are demo fixtures
Hallucination	Planned claim-level verifier labelling Verified / Uncertain / Unsupported, with human review	Review UI built; verifier not started (Phase IV/V)
Unsafe inputs	Bounded ingestion, ZIP checks, IP-pinned GitHub fetch, SHA-256 write-once storage	Implemented and tested
Over-confident structure	Four-state resolution: confirmed, ambiguous, unresolved, external	Implemented
Weak evaluation	Precision/recall/F1 scorer with frozen candidates and blank label forms	Framework verified; gold labels = 0 (REVIEW_PENDING)
Separate visualization	One dashboard with shared VizModel, provenance on every chart	Implemented for M1–M4; M5 partly

CHAPTER 3
PROPOSED SYSTEM

CodeAtlas AI treats a repository as two things at once: a program, with symbols and dependencies, and a text corpus, with tokens, identifiers and documentation. This chapter describes the design that serves both views. It covers the architecture, the methodology for each phase, the modules and the database.

3.1 DETAILED DESIGN

The system follows a three-tier architecture (Fig 3.1).

- **Presentation tier.** A React 18 and TypeScript single-page application. Every page obtains its data through one `DashboardApi` interface, which has two adapters. The Demo adapter serves a seeded, clearly labelled illustrative fixture. The Live adapter calls the backend API.
- **Application tier.** A FastAPI service that validates input, persists immutable snapshots and schedules background jobs. A Celery worker runs the preprocessing, structure and exploration analyzers.
- **Data tier.** PostgreSQL for metadata and analysis results, Redis as the job broker, and a content-addressed filesystem blob store for the original bytes.

The NLP model components for Phases IV and V (`nlp/`, `documentation_engine/`, `experiments/`) exist only as documented placeholders. They are drawn with dashed outlines in Fig 3.1 to show that they are not implemented.

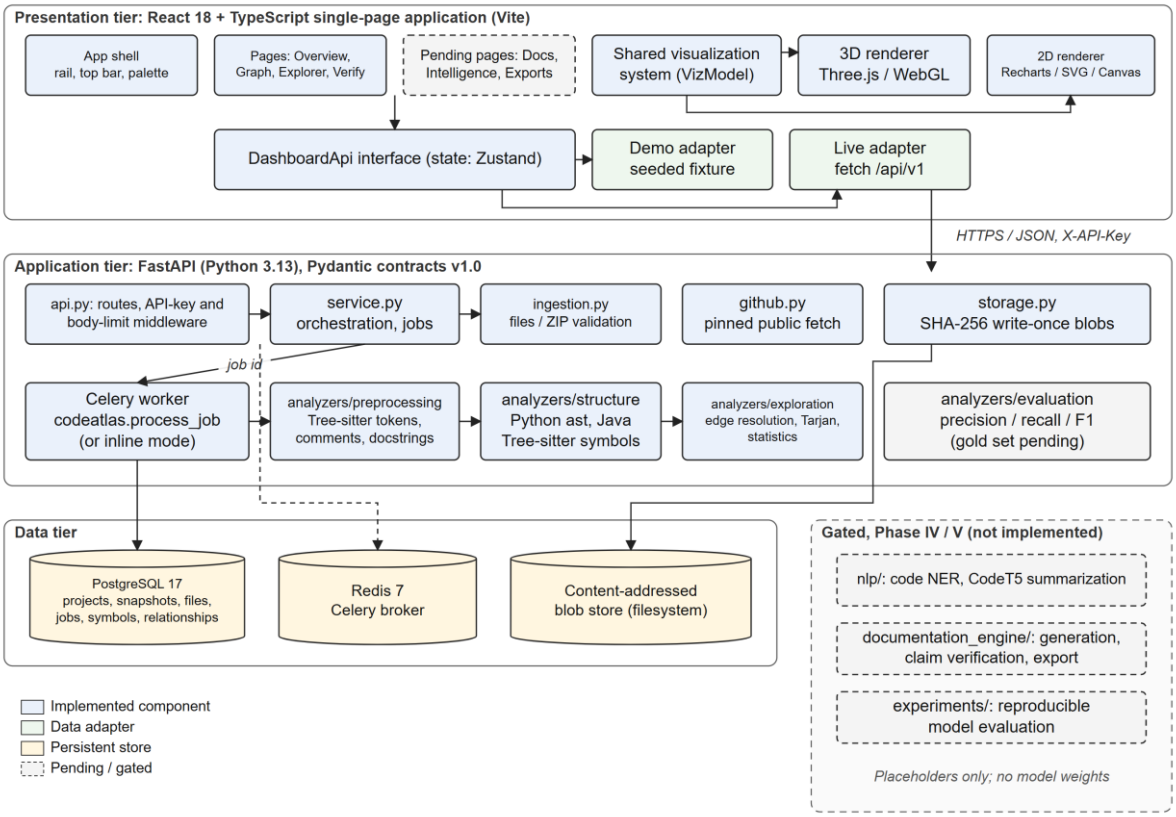


Fig 3.1 System Architecture of CodeAtlas AI

Table 3.1 Technology Stack

Layer	Technology (pinned version)	Purpose
Backend language	Python 3.13	Analysis pipeline and API
Web framework	FastAPI 0.162.0, Pydantic 2.14.2	REST API, versioned data contracts (schema 1.0)
Persistence	SQLAlchemy 2.1.6, Alembic 1.20.1, PostgreSQL 17	ORM, migrations, relational store
Jobs	Celery 5.6.3, Redis 7.4	Background preprocessing and analysis
Parsing	tree-sitter 0.25.2 (Python 0.25.0, Java 0.23.5), Python ast	Lexical tokens, Java structure, Python structure
Backend testing	pytest 9.2.0, Hypothesis 6.170, ruff 0.17.3	Unit, property-based and lint checks
Frontend	React 18.3.1, TypeScript 5.9.3, Vite 8.3.5, Tailwind 3.4	Single-page dashboard
Visualization	three 0.187.0, Recharts 3.10.1, d3-force 3.0.0	3D charts, 2D charts, graph layout
Interaction	Zustand 5, React Router 7.18, cmdk 1.1, Shiki 4.5, Framer Motion	State, routing, command palette, syntax highlighting, motion
Frontend testing	Vitest 5.0.5, Playwright 1.64.1, axe-core 4.14	Unit, browser and accessibility tests
Deployment	Docker Compose (postgres, redis, migrate, api, worker)	Reproducible local stack

Use cases. Fig 3.2 shows the actors and use cases. A *Developer* imports code, starts jobs and explores the results. A *Reviewer* inspects claims against evidence and approves, rejects or

marks them uncertain. An *Operator* runs the stack, and the *Celery worker* is a system actor that executes jobs. Documentation generation and export are drawn dashed because they belong to Phases IV and V.

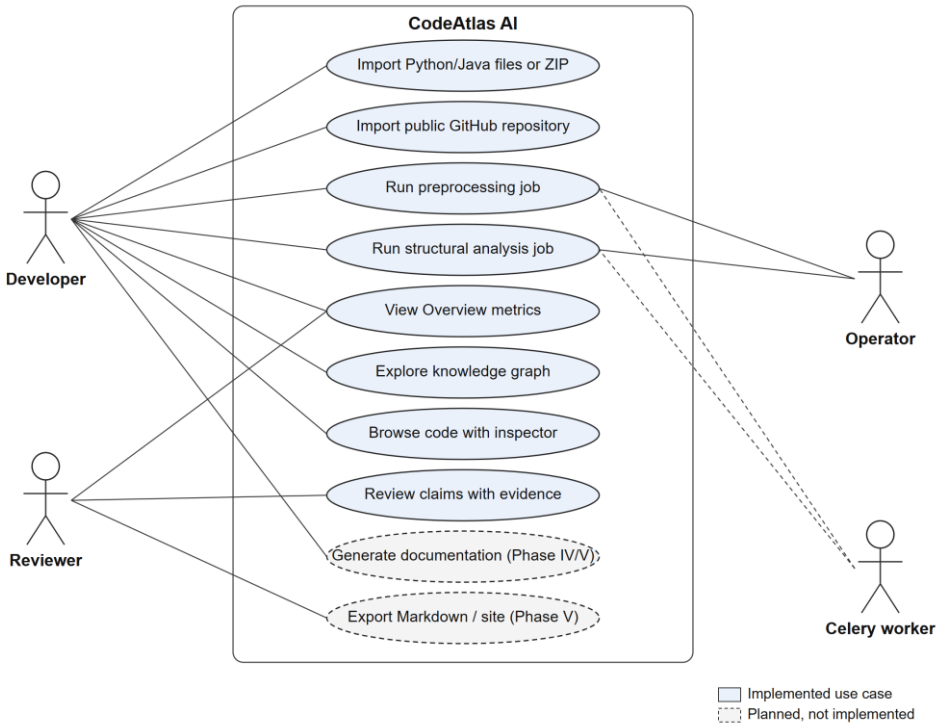


Fig 3.2 Use Case Diagram

Data flow. The context-level data flow diagram (Fig 3.3) treats the system as one process exchanging data with the developer, GitHub and the reviewer. The level-1 diagram (Fig 3.4) decomposes it into six processes and four data stores.

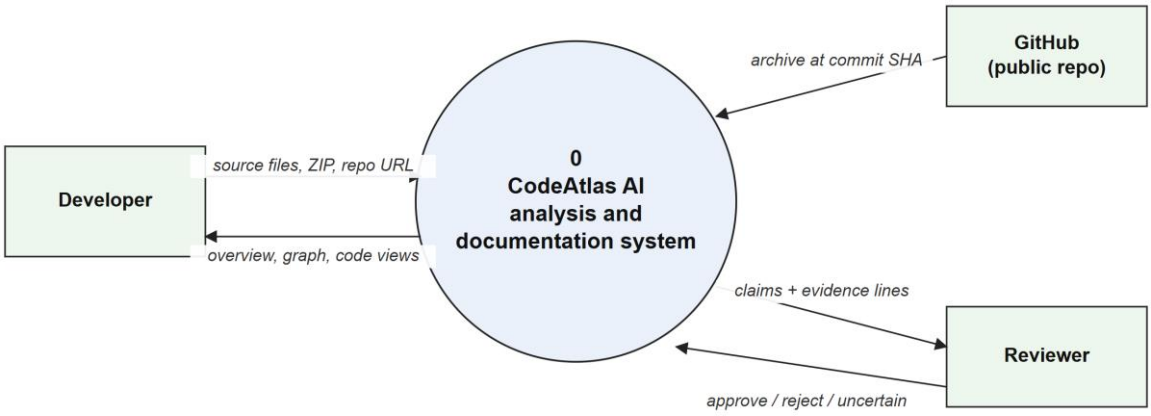


Fig 3.3 Data Flow Diagram, Level 0 (Context Diagram)

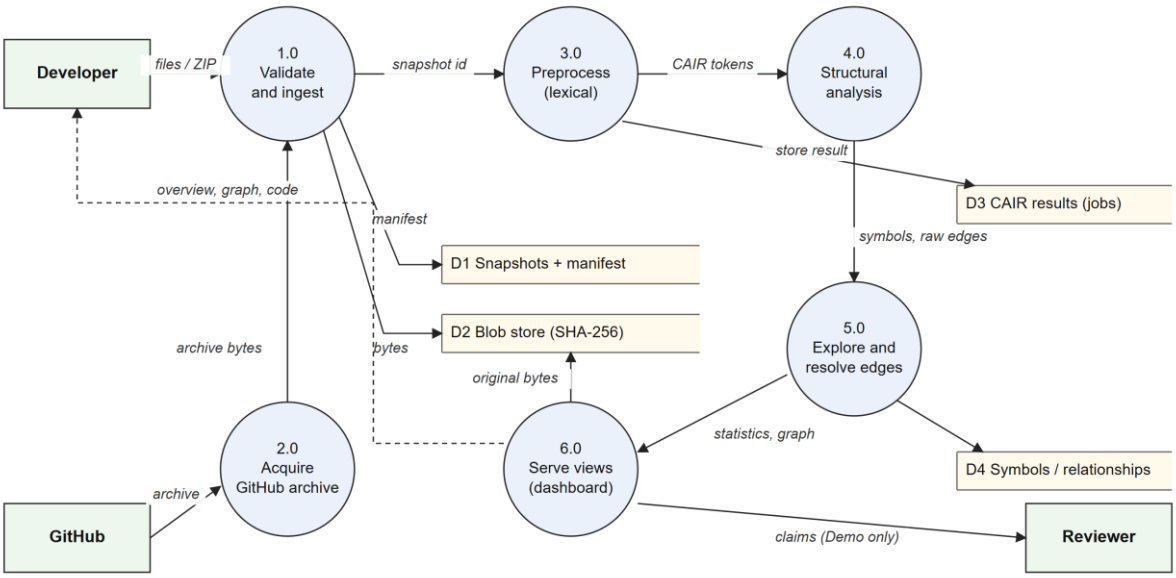


Fig 3.4 Data Flow Diagram, Level 1

Frontend data layer. The dashboard never mixes demo data with measured data. Fig 3.5 shows the single interface and its two adapters. If the backend is unreachable or an endpoint does not exist yet, the Live adapter returns a typed error (`service_unreachable`, `invalid_response`, `unauthorized`, `integration_pending` or `analysis_job_required`). The page then shows that error instead of quietly switching to demo data.

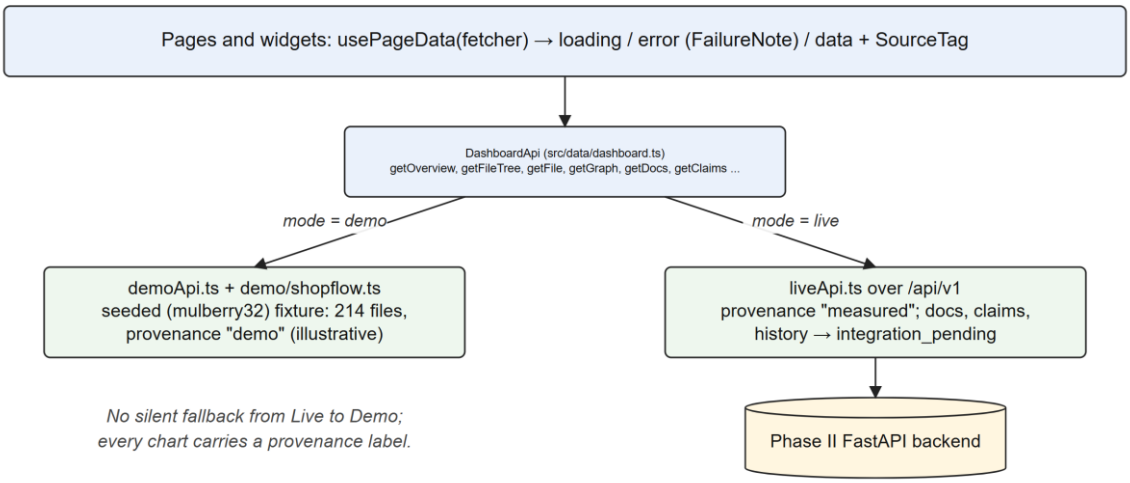


Fig 3.5 Frontend Data Layer with Demo and Live Adapters

Visualization system. All charts are produced from one normalized `VizModel`, and each can be rendered by a 3D WebGL renderer or a 2D SVG/Canvas renderer (Fig 3.6). One shared off-screen WebGL renderer draws each 3D chart and copies the result into that chart's canvas. This

keeps 22 chart stages within a single WebGL context, below the limit browsers place on the number of contexts per page.

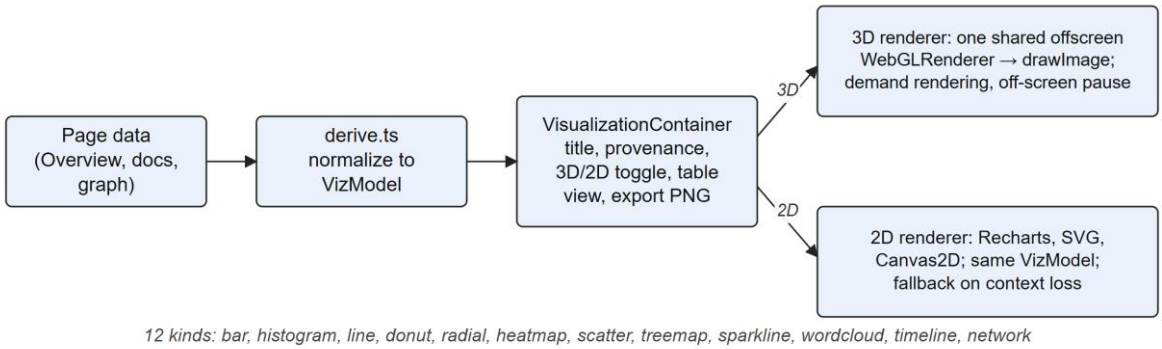


Fig 3.6 Shared 3D/2D Visualization Pipeline

3.2 METHODOLOGY

Development follows a gated, phased methodology (Fig 3.7). Each phase has written objectives, a definition of done and an acceptance record, and a later phase may not claim capabilities that an earlier gate has not verified. The status shown is taken from the project status record dated 3 November 2026.

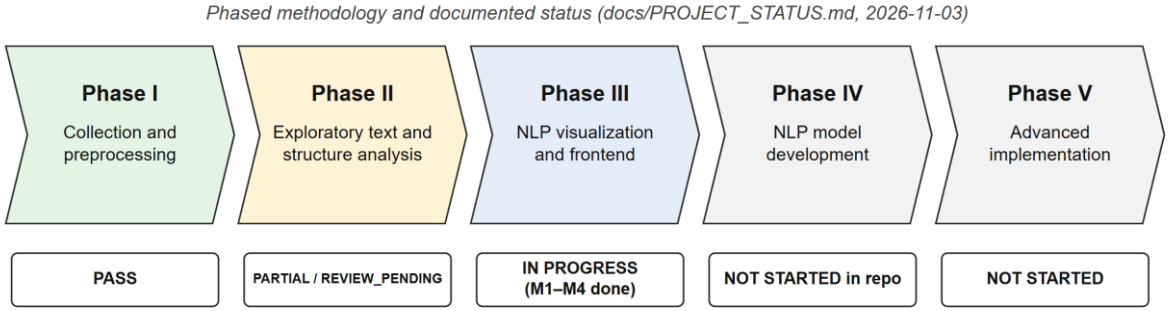


Fig 3.7 Phased Methodology and Documented Status

3.2.1 Collection and Source Preservation (Phase I)

Input arrives as individual files, a ZIP archive or a public GitHub URL. Only `.py` and `.java` files are analysed; other files are recorded as ignored in the import manifest. All limits are explicit and configurable: 500 files, 2 MiB per file, 20 MiB in total, 25 MiB per download, a 200:1 ZIP compression ratio, 100,000 tokens per file and a 15-second download deadline. ZIP archives are inspected through their central directory before any entry is read, and nothing is extracted to the filesystem. Entries with traversal paths (`../`) or symbolic links are rejected.

Python files are decoded according to PEP 263 encoding declarations. Java files are accepted as UTF-8 or as BOM-marked UTF-16. Every file is hashed with SHA-256 and written once to

a content-addressed blob store. A snapshot identifier is derived from the project and the content hash, so importing the same bytes into the same project yields the same snapshot. GitHub imports use the repository API [40] to resolve the default branch to a commit SHA, fetch from an allowlisted host over validated and pinned public IP addresses, and reject every redirect. Fig 3.8 shows the flow.

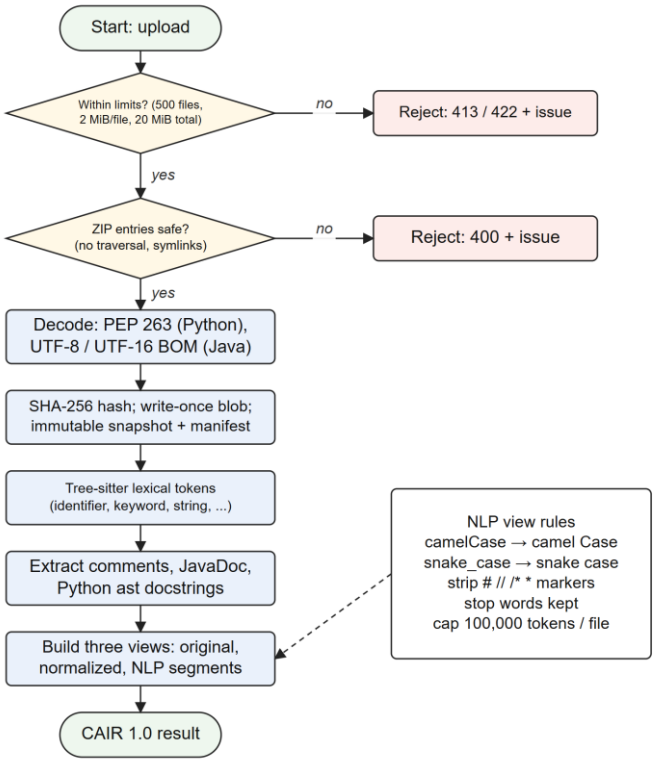


Fig 3.8 Flowchart of Ingestion and Preprocessing

3.2.2 Lexical Preprocessing into Three Views

Each file is tokenized with the Tree-sitter grammar for its language [29]. Every token is classified as identifier, keyword, string, number, operator, punctuation, comment or other. Comments and JavaDoc blocks come from the syntax tree. Python docstrings for modules, classes and functions (including nested ones) come from `ast` [45]. Code is never executed or evaluated. Positions use 1-based lines and 0-based Unicode code-point columns with half-open spans, so every token can be mapped back to the exact original bytes.

Three aligned views are produced. The *original* view is the exact decoded text. The *normalized* view changes only line endings. The *NLP* view applies the transformations of Algorithm 3.1.

```
ALGORITHM 3.1  NLP view of a token
Input : token text t, token kind k, configuration C
Output: normalized text for NLP analysis
1  if k = identifier and C.split_identifiers then
2      t <- insert a space at every boundary matching ([a-z0-9])([A-Z])
3      t <- replace every "_" in t with a space
4  else if k in {comment, javadoc} and C.filter_comment_markers then
5      t <- remove leading markers  #  //  /*  */  *  at each line start
6      t <- remove a trailing */
7  t <- collapse every run of whitespace in t to a single space
8  return t                      // stop words are deliberately kept
```

3.2.3 Structure Extraction (Phase II)

Python structure is read with `ast.parse`. Java structure is read by traversing Tree-sitter declarations. The analyzer emits symbols of eleven kinds (module, class, interface, enum, record, function, method, constructor, variable, field and parameter). Each symbol has a signature, parameters, modifiers, decorators, a documented flag and an evidence span. Relationships of five kinds (contains, imports, inherits, implements and calls) are emitted with the span of the statement that produced them. HTTP endpoints are recognised only for literal FastAPI route paths and Spring `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping` and `@DeleteMapping` annotations.

3.2.4 Conservative Relationship Resolution

Static analysis of Python cannot know which object a name refers to at run time, and Java analysis without type resolution has the same difficulty. Instead of guessing, CodeAtlas AI assigns every relationship one of four resolution states using the rules in Fig 3.9. A link is *confirmed* only when exactly one candidate exists. Several candidates make it *ambiguous*. A missing relative import is *unresolved*, and a missing absolute import is *external* (for example, a standard-library module). Fan-in and fan-out are computed only from confirmed import edges. Import cycles are the strongly connected components of the confirmed module graph, found with Tarjan's algorithm [30] in O(V + E) time.

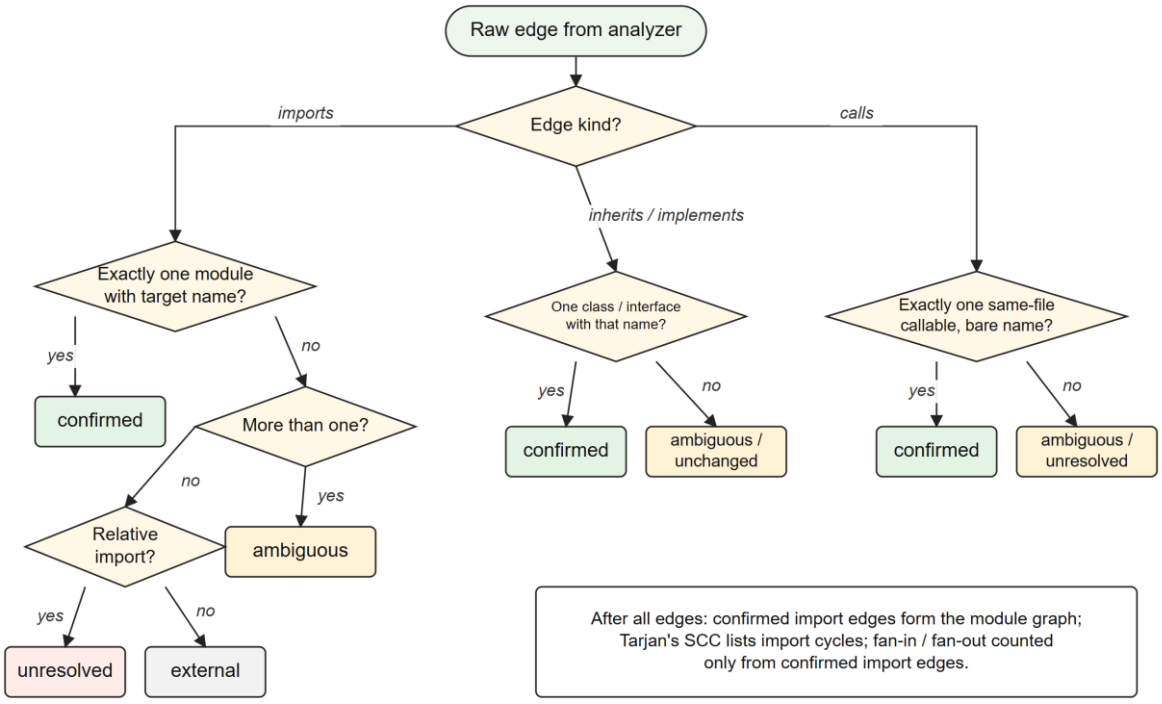


Fig 3.9 Flowchart of Relationship Resolution

3.2.5 Exploratory Text Statistics

Let N be the number of non-comment tokens in a snapshot and V the set of their distinct spellings. Vocabulary size and lexical diversity (the type–token ratio [35]) are defined as:

$$vocabulary_size = |V|, \quad lexical_diversity = |V| / N \quad (0 \text{ when } N = 0) \tag{3.1}$$

Let S_d be the set of *documentable* symbols (classes, interfaces, enums, records, functions, methods and constructors), and $D \subseteq S_d$ those that carry a docstring or JavaDoc. Documentation coverage is:

$$coverage = |D| / |S_d| \tag{3.2}$$

For a module m in the confirmed import graph G = (M, E):

$$fan_in(m) = |\{ (u, m) \in E \}|, \quad fan_out(m) = |\{ (m, v) \in E \}| \tag{3.3}$$

Table 3.2 lists every statistic in the `ExploratoryStatistics` contract and how it is computed. All of them are deterministic: the same snapshot always produces the same numbers.

Table 3.2 Exploratory Statistics and Their Computation

Statistic	Definition / computation
tokens, token_types	All lexical records; counts per token kind
token_frequencies, identifiers, keywords	Frequency lists sorted by count (descending), then value
vocabulary_size, lexical_diversity	Equation 3.1 over non-comment tokens

Statistic	Definition / computation
documentation_terms	Lower-cased matches of [A-Za-z][A-Za-z0-9]* in comments, docstrings and JavaDoc; no stop-word filtering
identifier_lengths	Histogram of identifier lengths in characters
naming_patterns	Per identifier: contains "_" → snake_case; matches [a-z][A-Z] → camelCase; all upper → UPPERCASE; else other
comment_lines, comment_lengths, docstring_lengths	Inclusive line spans and length distributions of comments and docstrings
documented_symbols, documentable_symbols, documentation_coverage	Equation 3.2; also per kind (documentation_by_kind) and the list of undocumented symbols
symbols_by_kind, functions_per_module, methods_per_class	Counts over extracted symbols
function_lengths, class_sizes	Inclusive declaration line spans
relationships_by_status	Counts of confirmed / ambiguous / unresolved / external edges
module_fan_in, module_fan_out, circular_dependencies	Equation 3.3; Tarjan SCCs of the confirmed import graph
files, languages, source_lines, duplicate_files	Manifest counts; duplicates detected by identical SHA-256

3.2.6 Evaluation Framework

Structure extraction is evaluated against human annotations. The evaluation corpus contains seven synthetic source files (four Python, three Java) with frozen candidate lists of 42 symbols and 50 relationships, together with blank label forms for independent annotators. The scorer compares predicted items P with gold items G using Equations 3.4 and 3.5:

$$precision = |P \cap G| / |P|, \quad recall = |P \cap G| / |G| \tag{3.4}$$

$$F1 = 2 \cdot precision \cdot recall / (precision + recall) \tag{3.5}$$

The scorer is implemented and its unit tests pass. No independent gold annotations have been collected yet, so precision, recall and F1 are recorded as *null* and Phase II remains REVIEW_PENDING. No metric value is claimed in this report.

3.2.7 Visualization and Interaction Design

The dashboard follows the "overview first, zoom and filter, then details on demand" principle [34]. The knowledge graph uses a force-directed layout [32] implemented with d3-force [33]. The forces are link springs with a distance that depends on the edge kind, many-body repulsion, a pull toward the centre of each node's cluster, and collision avoidance. The parameters are listed in Table 3.3. Node radius grows with the square root of degree, so that hub modules stand out without hiding smaller ones:

$$r(v) = 2.6 + 1.7 \cdot \sqrt{deg(v)} \quad (3.6)$$

Labels are drawn in order of node degree, up to a budget that depends on the zoom factor z (14 labels for z < 0.45, 24 for z < 0.8, 70 for z < 1.4, 180 for z < 2.2, and all labels above that). In performance mode the budget is halved. Cluster outlines are convex hulls computed with Andrew's monotone-chain algorithm [31].

Table 3.3 Force-Directed Layout Parameters (Knowledge Graph)

Parameter	Value
Link distance by edge kind	contains 18, calls 46, inherits 52, imports 60
Cross-cluster link strength	Multiplied by 0.15
Many-body charge	−charge, user slider 80–600; distanceMax 420; theta 0.95
Cluster centring	forceX / forceY toward cluster centre, strength 0.14
Collision	Node radius plus padding
Cooling	alphaDecay 0.028; layout settles in 12 ms slices until alpha < 0.02
Automatic performance mode	When the median of 30 frame gaps exceeds 24 ms on two consecutive checks

3.2.8 Claim Review Workflow

In the target system, Phase IV will generate claims about each symbol and a verifier will label each claim Verified, Uncertain or Unsupported. The review workflow (Fig 3.10) is already built. A rule-based evidence locator scans the symbol's own lines for text bearing on the claim (for example `raise X` for a claim that a function raises X, or `.commit()` as contradicting evidence for a claim that a function writes inside the caller's transaction). The reviewer then approves (A), rejects (R) or marks the claim uncertain (U). The locator only points the reviewer at lines: it does not decide a claim's status, and a match is not proof [26], [28]. Until Phase IV exists, the claims shown are from the demo fixture and are labelled as illustrative.

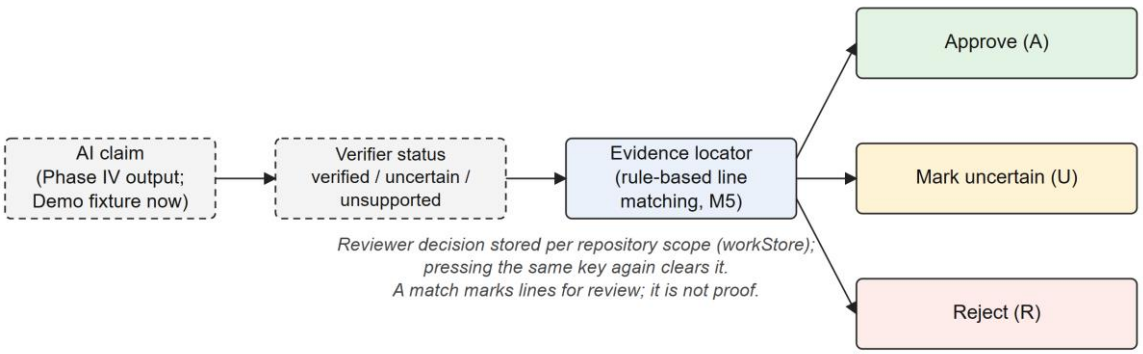


Fig 3.10 Claim Review Workflow

Interaction sequences. Fig 3.11 shows the sequence of an import followed by preprocessing and analysis jobs. Preprocessing and analysis job identifiers are derived from the snapshot, job kind and configuration, so repeating a request returns the existing job (GitHub import jobs use random identifiers). The message queue carries only the job identifier; the worker reads everything else from the database under a row lock. Fig 3.12 shows how a dashboard page loads data in Live mode, including the explicit failure path for features that are not yet integrated.

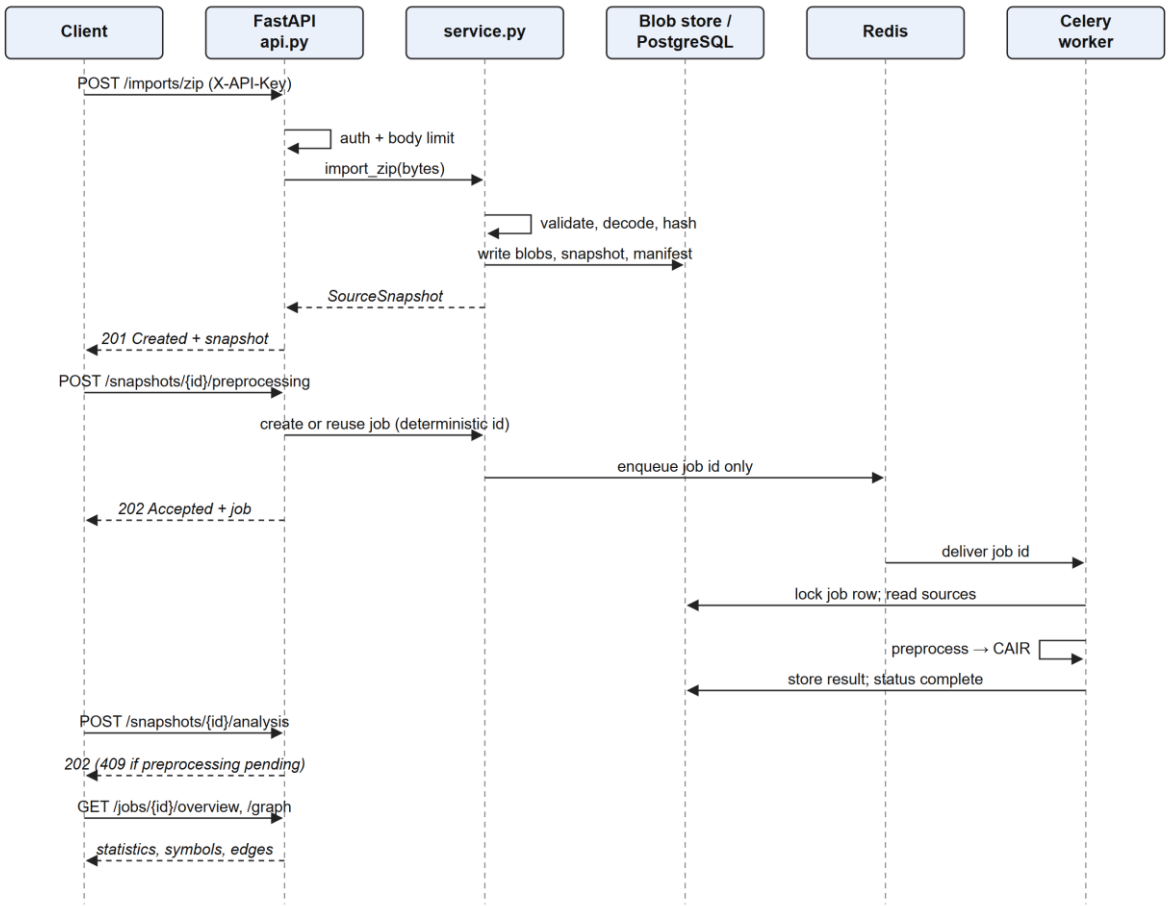


Fig 3.11 Sequence Diagram: Import, Preprocessing and Analysis

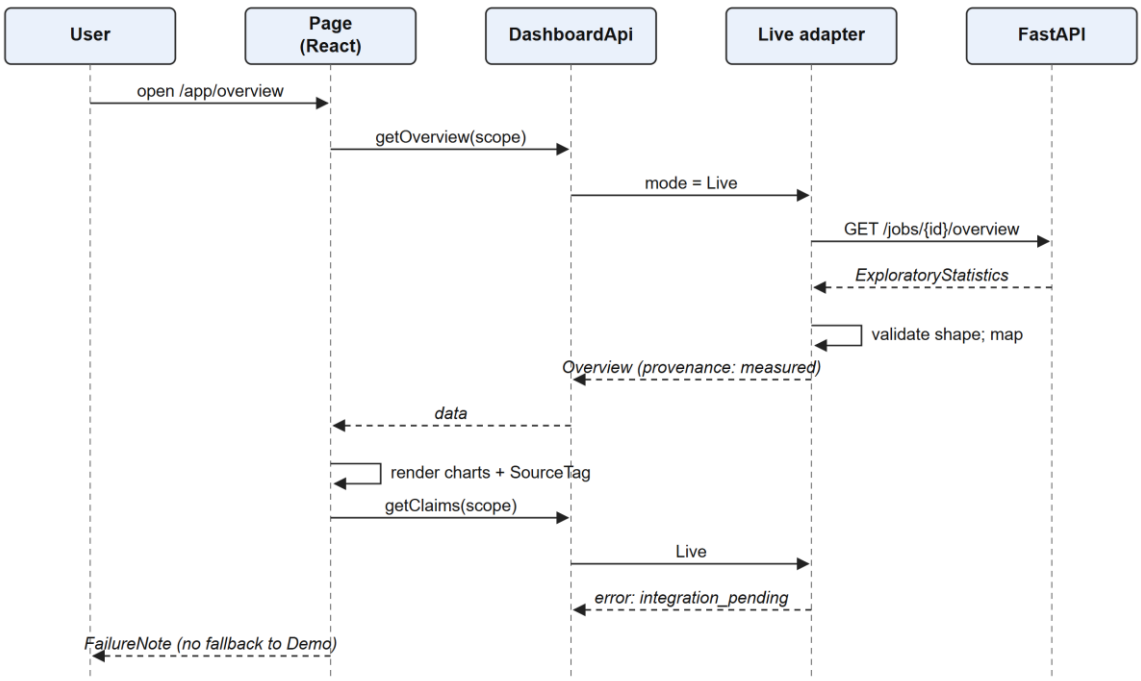


Fig 3.12 Sequence Diagram: Dashboard Page in Live Mode

3.3 MODULES

The system is divided into the modules below. Each table lists the module's purpose, input, processing, output, source files, interactions and verified status.

Table 3.4 Module 1: Ingestion and Validation

Attribute	Description
Purpose	Accept files and ZIP archives safely
Input	Multipart files or a ZIP body; project name
Processing	Size, count and ratio limits; central-directory inspection; traversal and symlink rejection; encoding detection; noise-directory filtering
Output	Validated source files and import manifest with ignored files and issues
Files	`backend/codeatlas/ingestion.py`, `config.py`
Interaction	Called by the service layer; writes through the storage module
Status	Implemented; requirements ING-01 to ING-09 verified

Table 3.5 Module 2: GitHub Acquisition

Attribute	Description
Purpose	Import a public GitHub repository reproducibly
Input	Canonical github.com repository URL
Processing	Resolve the default branch to a commit SHA; allowlisted host; DNS answers validated as public and pinned with SNI; proxies disabled; redirects rejected; bounded download
Output	Archive bytes at a fixed commit, passed to ingestion

Attribute	Description
Files	`backend/codeatlas/github.py`
Interaction	Runs as a background job of kind github
Status	Implemented and unit-tested (GH-01 to GH-06); a live GitHub run has not been executed

Table 3.6 Module 3: Content-Addressed Storage

Attribute	Description
Purpose	Preserve original bytes exactly
Input	Decoded and raw file bytes
Processing	SHA-256 hashing; write-once blob files; tamper detection on read
Output	Blob paths keyed by hash; byte-identical originals served with ETag
Files	`backend/codeatlas/storage.py`
Interaction	Used by ingestion and by the original-file endpoint
Status	Implemented (SRC-01 to SRC-05); relies on POSIX file flags, so native Windows is unsupported

Table 3.7 Module 4: Lexical Preprocessing

Attribute	Description
Purpose	Turn source into NLP-ready text views
Input	Snapshot source files and PreprocessingConfig (phase1-v1)
Processing	Tree-sitter tokenization; comment, JavaDoc and docstring extraction; Algorithm 3.1; Unicode spans; per-file token cap; broken-syntax warnings
Output	CAIR 1.0 result: three views, tokens, comments, NLP segments, statistics, issues
Files	`analyzers/preprocessing.py`
Interaction	Executed by the worker for preprocess jobs
Status	Implemented (PRE-01 to PRE-10)

Table 3.8 Module 5: Structure Extraction

Attribute	Description
Purpose	Extract symbols, relationships and endpoints
Input	CAIR results
Processing	Python ast and Java Tree-sitter traversal; signatures, decorators, documented flag; literal route extraction
Output	Symbols (11 kinds), raw relationships (5 kinds), endpoint declarations
Files	`analyzers/structure.py`
Interaction	Called by the exploration module
Status	Implemented; empirical accuracy pending gold labels

Table 3.9 Module 6: Exploration and Statistics

Attribute	Description
Purpose	Resolve edges and compute corpus statistics
Input	Symbols, raw edges, tokens and comments
Processing	Rules of Fig 3.9; Tarjan SCC; the statistics of Table 3.2
Output	StructuralAnalysis with ExploratoryStatistics and DependencyGraph
Files	`analyzers/exploration.py`
Interaction	Persists results to analysis tables; served by the overview and graph endpoints
Status	Implemented (ETA-01 to ETA-05)

Table 3.10 Module 7: Extraction Evaluation

Attribute	Description
Purpose	Measure extraction accuracy against human labels
Input	Frozen candidates and completed label forms
Processing	True positive, false positive and false negative matching; Equations 3.4–3.5
Output	Precision, recall and F1 per item type
Files	`analyzers/evaluation.py`, `scripts/extraction_evaluation.py`
Interaction	Offline script
Status	Scorer verified by tests; gold labels = 0, metrics null (REVIEW_PENDING)

Table 3.11 Module 8: API, Jobs and Orchestration

Attribute	Description
Purpose	Expose the pipeline through a versioned REST API
Input	HTTP requests with optional X-API-Key
Processing	Constant-time key comparison; body limits; error envelopes; deterministic job ids; Celery dispatch with acks_late, prefetch 1 and time limits; inline mode for demos
Output	19 operations under /api/v1; job status and results
Files	`api.py`, `service.py`, `main.py`, `workers/tasks.py`
Interaction	Used by the Live adapter and test clients
Status	Implemented (API-01 to API-10, JOB-01)

Table 3.12 Module 9: Frontend Data Layer

Attribute	Description
Purpose	Provide pages with typed data and clear provenance
Input	Data mode (Demo or Live), repository scope
Processing	DashboardApi with Demo (seeded fixture) and Live (/api/v1 mapping, response validation, typed errors) adapters
Output	Typed page data with provenance "demo" or "measured"
Files	`frontend/src/data/`
Interaction	Used by every page through usePageData

Attribute	Description
Status	Implemented (M2); Live docs, claims and history are integration pending

Table 3.13 Module 10: Shared Visualization System

Attribute	Description
Purpose	Render every chart consistently in 3D and 2D
Input	VizModel and provenance
Processing	12 chart kinds; shared off-screen WebGL renderer; demand rendering; table view; PNG export; reduced-motion support
Output	Interactive charts inside a VisualizationContainer
Files	`frontend/src/viz/`
Interaction	Used by the Overview, Graph, Verification and design-system pages
Status	Implemented (M1, accepted)

Table 3.14 Module 11: Overview Page

Attribute	Description
Purpose	Summarise repository health
Input	Overview, docs report and analysis status
Processing	KPI strip, coverage trend, language and size charts, completeness ring (unmeasured stages labelled Phase IV)
Output	Overview dashboard
Files	`frontend/src/app/pages/overview/`
Interaction	Links to the Graph and Explorer pages
Status	Implemented (M3)

Table 3.15 Module 12: Knowledge Graph

Attribute	Description
Purpose	Explore modules, classes and their relationships
Input	DashboardGraph (nodes, edges, clusters)
Processing	Force layout (Table 3.3); 3D and 2D renderers; search ranking; filters; label level of detail; frame statistics; minimap; PNG export
Output	Interactive graph with inspector
Files	`frontend/src/app/pages/graph/`
Interaction	Opens files in the Explorer
Status	Implemented (M4)

Table 3.16 Module 13: Code Explorer

Attribute	Description
Purpose	Read code with structure and documentation alongside
Input	File tree, file document, symbols and relations

Attribute	Description
Processing	Accessible tree with filter; Shiki syntax highlighting; documentation gutter badges; hover card; inspector with Explanation, Signature, Relationships and Docs tabs; resizable panes
Output	Three-pane explorer
Files	`frontend/src/app/pages/explorer/`, `pages/code/`
Interaction	Deep-linked from the Graph page and command palette
Status	Built and checked in the browser; automated tests not yet written

Table 3.17 Module 14: Verification Workspace

Attribute	Description
Purpose	Review claims against source evidence
Input	Claims, file text and symbol spans
Processing	Status summary and charts; windowed claim queue; J/K/A/R/U keys; rule-based evidence locator; support and contradict line marks; SVG connector
Output	Reviewer decisions per repository scope
Files	`frontend/src/app/pages/verification/`
Interaction	Uses the shared code view
Status	Built on demo claims; Live claims integration pending; automated tests not yet written

Table 3.18 Module 15: Pending Modules

Attribute	Description
Purpose	Docs Studio, Intelligence, Exports, marketing site; NLP models
Input	Not applicable
Processing	Not implemented
Output	Not applicable
Files	`nlp/`, `documentation_engine/`, `experiments/` (README placeholders)
Interaction	Not applicable
Status	NOT STARTED (Phase III M5–M7, Phases IV and V)

3.4 DATABASE DESIGN

The relational schema is deliberately small. Each table keeps its identifying and indexed columns as real columns and stores the rest of the versioned contract as a JSON payload, so the contract can evolve without a migration for every field. Two Alembic migrations create the schema (`0001_phase_one`, `0002_phase_two`). PostgreSQL is used in deployment; SQLite is used only in tests and corpus reports. Original file bytes are kept outside the database in the blob store.

3.4.1 ER Diagram

Fig 3.13 shows the entities and their relationships. A project has many snapshots. A snapshot has many source files and many jobs. An analysis job produces many symbols and relationships, and each symbol belongs to a source file.

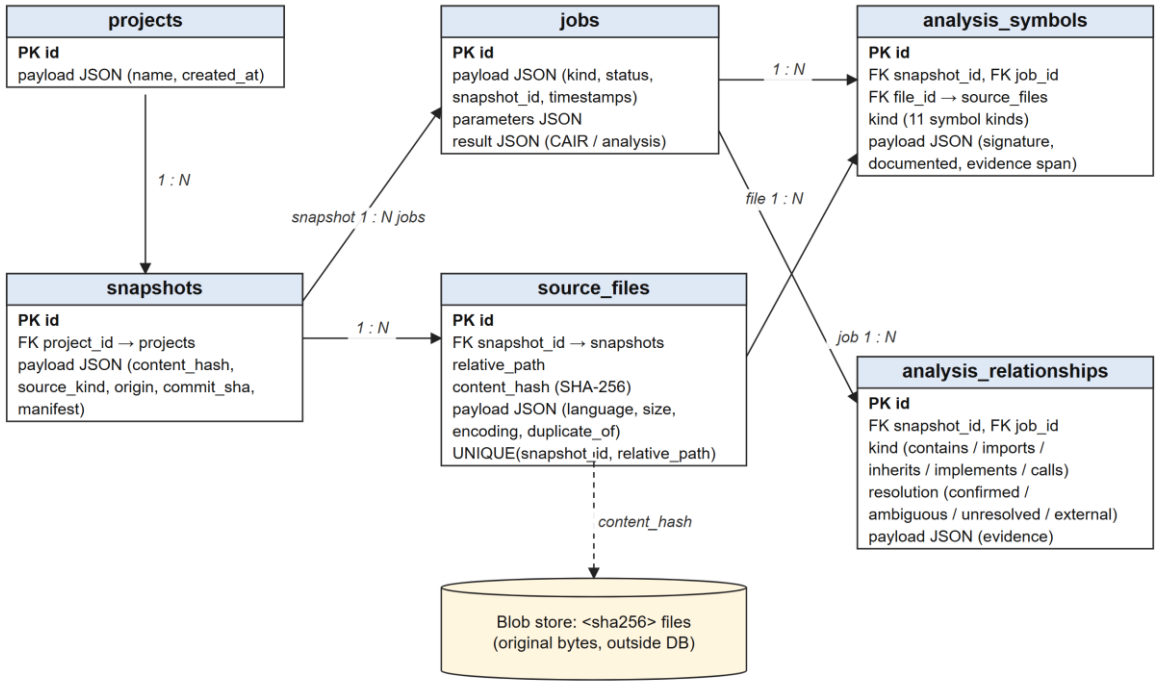


Fig 3.13 Entity–Relationship Diagram

3.4.2 Tables or Entities

Table 3.19 Structure of the projects Table

Column	Type	Constraint	Description
id	string	Primary key	Project identifier
payload	JSON	Not null	Project contract: name, created_at

Table 3.20 Structure of the snapshots Table

Column	Type	Constraint	Description
id	string	Primary key	Stable identifier from project id and content hash
project_id	string	Foreign key → projects.id	Owning project
payload	JSON	Not null	content_hash, source_kind (files / zip / github), origin, commit_sha, manifest

Table 3.21 Structure of the source_files Table

Column	Type	Constraint	Description
id	string	Primary key	File identifier
snapshot_id	string	Foreign key → snapshots.id	Owning snapshot
relative_path	string	Unique with snapshot_id	Normalized path inside the snapshot
content_hash	string	Not null	SHA-256 of the original bytes (blob key)
payload	JSON	Not null	language, size_bytes, encoding, duplicate_of, analysis_status

Table 3.22 Structure of the jobs Table

Column	Type	Constraint	Description
id	string	Primary key	Deterministic (snapshot + kind + configuration) for preprocess and analyze jobs; random for GitHub jobs
payload	JSON	Not null	kind (preprocess / github / analyze), status (queued / running / complete / failed), timestamps
parameters	JSON	Not null	Job configuration
result	JSON	Nullable	CAIR or StructuralAnalysis

Table 3.23 Structure of the analysis_symbols Table

Column	Type	Constraint	Description
id	string	Primary key	Symbol identifier
snapshot_id, job_id	string	Foreign keys	Snapshot and producing analysis job
file_id	string	Foreign key → source_files.id	Declaring file
kind	string	Indexed	One of 11 symbol kinds
payload	JSON	Not null	name, qualified_name, signature, parameters, modifiers, decorators, documented, evidence span

Table 3.24 Structure of the analysis_relationships Table

Column	Type	Constraint	Description
id	string	Primary key	Relationship identifier
snapshot_id, job_id	string	Foreign keys	Snapshot and producing analysis job
kind	string	Indexed	contains, imports, inherits, implements, calls
resolution	string	Indexed	confirmed, ambiguous, unresolved, external
payload	JSON	Not null	source_id, target_id or target_name, evidence span

CHAPTER 4
IMPLEMENTATION

This chapter describes how the modules of Chapter 3 were implemented, presents the results they produce, and reports the testing performed. Commit history shows the order of the work: initial setup and Phase I, then Phase II, then Phase III milestones M1 (visualization system), M2 (dashboard shell and Demo/Live data), M3 (Overview), M4 (Knowledge Graph), and the Explorer and Verification pages. The backend comprises about 2,800 lines of Python, of which the analyzers account for 1,240.

4.1 MODULE IMPLEMENTATION

4.1.1 Backend Service and API

The backend is a FastAPI [38] application created by a factory function in `backend/codeatlas/api.py`. Middleware enforces a request body limit and, when `CODEATLAS_API_KEY` is set, requires a matching `X-API-Key` header, compared in constant time with `hmac.compare_digest`. When no key is configured the service is intended only for loopback development use. Errors are returned in a uniform envelope with HTTP status codes 400, 404, 409, 413, 415, 422 and 503. Table 4.1 lists all 19 operations. The OpenAPI document is exported to `docs/03-architecture/openapi.json` by a script so that the frontend types can be generated from it.

Table 4.1 REST API Operations (all under /api/v1)

Method	Path	Purpose
GET	/health	Database readiness
GET	/projects, /projects/{id}	List projects; get a project
GET	/projects/{id}/snapshots	Snapshots of a project
POST	/imports/files?name=	Import uploaded files (201)
POST	/imports/zip	Import a ZIP archive (201)
POST	/imports/github	Start a GitHub import job (202)
GET	/snapshots/{id}, /snapshots/{id}/manifest	Snapshot and its import manifest
GET	/snapshots/{id}/files/{file_id}/original	Byte-identical original file with ETag
POST	/snapshots/{id}/preprocessing	Start preprocessing (202)
POST	/snapshots/{id}/analysis	Start structural analysis (202; 409 if preprocessing pending)
GET	/jobs/{id}	Job status

Method	Path	Purpose
GET	/jobs/{id}/results, /issues, /statistics	CAIR result, pipeline issues, preprocessing statistics
GET	/jobs/{id}/analysis, /graph, /overview	Structural analysis, dependency graph, exploratory statistics

Jobs run in a Celery [39] worker (`workers/tasks.py`, task `codeatlas.process_job`). The broker message carries only the job identifier, and serialization is JSON-only. Tasks are acknowledged late with a prefetch of one, and they have a soft time limit of 270 s and a hard limit of 300 s. A PostgreSQL row lock makes a duplicate delivery of the same job harmless. For demonstrations, `CODEATLAS_JOB_MODE=inline` runs the same pipeline synchronously inside the API process. Docker Compose defines the PostgreSQL, Redis, migration, API and worker services. The containers run as a non-root user with dropped capabilities and a read-only root filesystem.

4.1.2 Preprocessing and Analysis

`analyzers/preprocessing.py` walks the Tree-sitter syntax tree iteratively, so deeply nested files cannot overflow the Python call stack. It emits a `TokenRecord` for each leaf token and applies Algorithm 3.1 to build NLP segments (Appendix A, Listing A.1). `analyzers/structure.py` produces symbols and raw relationships. `analyzers/exploration.py` resolves relationships, finds cycles with Tarjan's algorithm (Listing A.2) and computes the statistics of Table 3.2 (Listing A.3).

The pipeline was run on the project's evaluation corpus: seven synthetic source files written for testing (four Python, three Java) released under the MIT licence. Table 4.2 gives the measured results. These numbers are produced by the implemented code on that small corpus. They describe the corpus; they are not a benchmark of extraction accuracy.

Table 4.2 Measured Results on the Evaluation Corpus (Phase II)

Measure	Value
Files analysed / languages	7 (4 Python, 3 Java)
Source lines	65
Tokens (all kinds)	205
Comments / docstrings	7 / 10
Unique identifier spellings	25
Vocabulary size / lexical diversity	59 / 0.2980
Documentation coverage	10 of 11 documentable symbols = 90.9%

Measure	Value
Symbols	20 (module 7, class 4, method 3, function 2, constructor 2, field 1, variable 1)
Relationships by resolution	20 (confirmed 13, external 4, unresolved 3, ambiguous 0)
Import cycles / endpoints	0 / 0
Duplicate files / ignored files / warnings / errors	2 / 1 / 5 / 0
Top identifiers	self 8, name 6, price 6, prices 6
Top documentation terms	price 7, a 5, and 5, catalog 5, synthetic 5, the 5

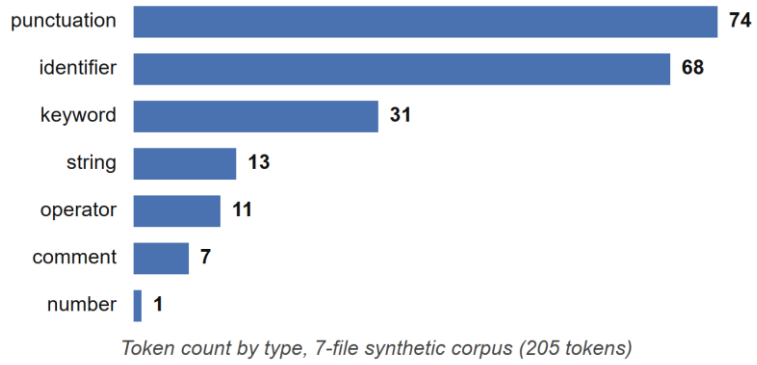


Fig 4.1 Token Distribution by Type on the Evaluation Corpus (Measured)

Fig 4.1 shows that punctuation and identifiers dominate the token stream, which is typical of source code. The top documentation terms include function words such as *a*, *and* and *the*, because stop words are deliberately kept (Section 2.3). Fig 4.2 and Fig 4.3 are the charts generated by the Phase II report script from the same analysis.

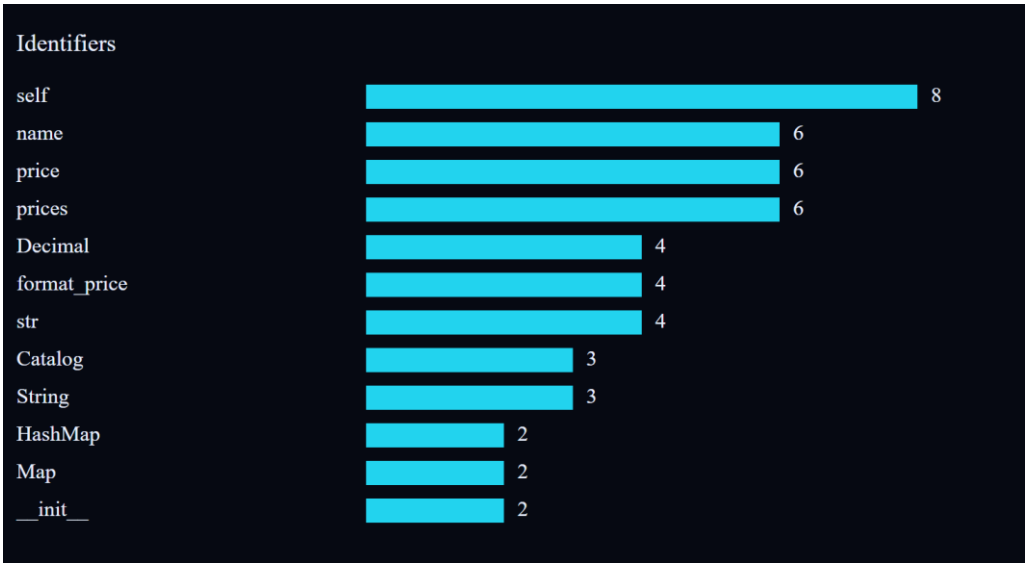


Fig 4.2 Identifier Frequencies Generated by the Phase II Report Script (Measured)



Fig 4.3 Documentation Coverage Chart Generated by the Phase II Report Script (Measured)

4.1.3 Frontend Shell and Data Layer

The frontend is a Vite build of a React 18 application in strict TypeScript. The shell provides a collapsible navigation rail, a top bar with repository and snapshot pickers, a data-mode switch (Demo or Live) and a 3D/2D chart preference, a command palette (`cmdk`), a keyboard-shortcut overlay, notifications, and Settings and Help pages. Every page loads data through `usePageData`, which renders a loading state, a `FailureNote` for typed errors, or the data together with a `SourceTag` naming its provenance.

The Demo adapter generates a repository called *shopflow-api* from a seeded random number generator (mulberry32), so every run shows the same data. It contains 214 files, 1,386 functions, 142 classes, a 703-node / 1,266-edge graph with 8 clusters and exactly three deliberate import cycles, coverage rising from 41% to 78%, and 3,912 claims (3,305 verified, 402 uncertain, 205 unsupported). **These are illustrative fixture values, not measurements of any real repository**, and the interface labels them "Demo, illustrative" wherever they appear. The Live adapter maps the Phase II endpoints onto the same types. It returns `integration_pending` for documentation, claims, notifications and coverage history, which the backend does not yet provide.

4.1.4 Visualization System (M1)

The visualization system supports twelve chart kinds, all driven by one `VizModel`. In 3D, a single off-screen Three.js `WebGLRenderer` [41] renders each chart and copies the result into that chart's canvas with `drawImage`. Rendering happens only when something changes, charts outside the viewport are paused, and an orthographic camera is used for charts whose values must be readable. If the WebGL context is lost, the chart falls back to 2D. In 2D the same models are drawn with Recharts, SVG or Canvas2D. Each chart sits in a `VisualizationContainer` that provides the provenance label, the 3D/2D toggle, an accessible table view and PNG export. Unit tests check data integrity by comparing the magnitudes drawn with the magnitudes supplied.

4.1.5 Overview and Knowledge Graph (M3, M4)

The Overview page (Fig 4.4) combines a KPI strip, coverage over time, distributions by language and module, and a completeness ring whose unmeasured stages are labelled "Phase IV" rather than given invented values. The Knowledge Graph (Fig 4.5) implements the force layout of Table 3.3 in both renderers. It supports search ranked by exact match, prefix, substring and then path (ties broken by degree), filters by node and edge kind, focus on a selected node and its neighbourhood, a minimap, label level of detail, and PNG export. A `FrameStats` helper measures p50 and p95 frame gaps and frames per second. Above 24 ms median frame gap it switches automatically to a performance mode, which halves the label budget and simplifies edge drawing.

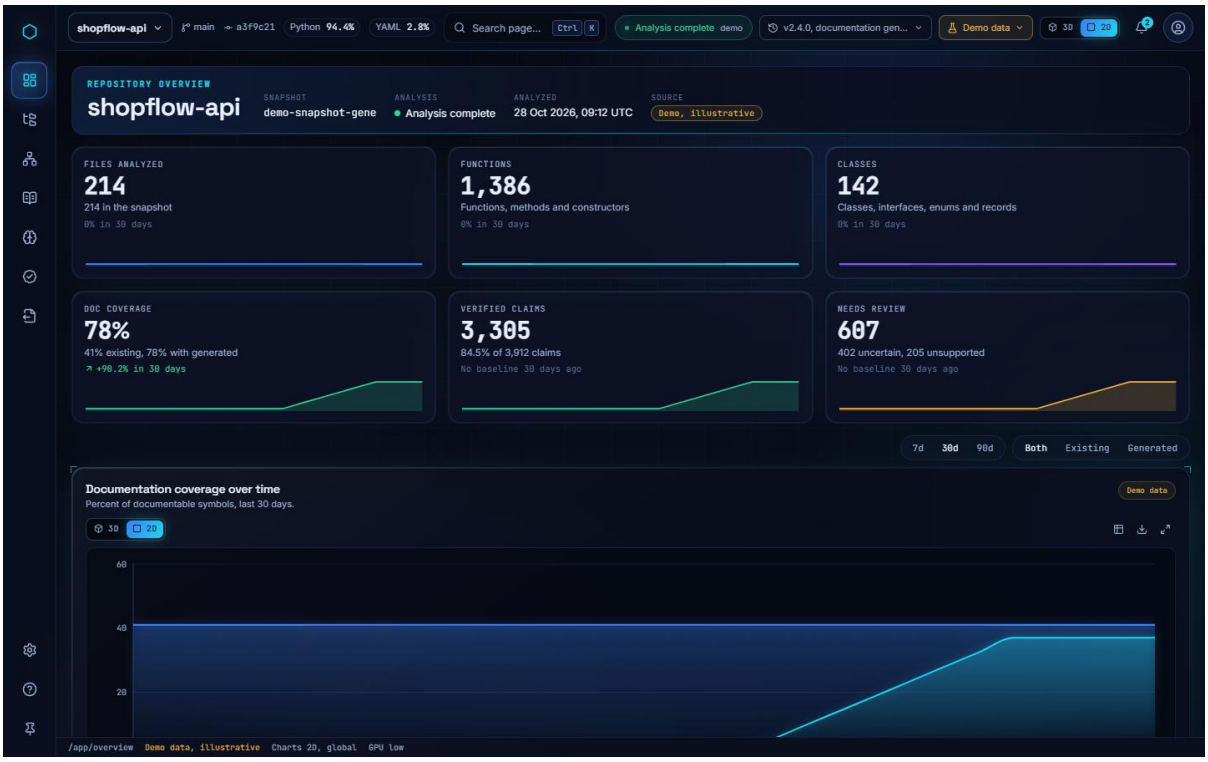


Fig 4.4 Overview Page in 2D Mode (Demo Data, Illustrative)

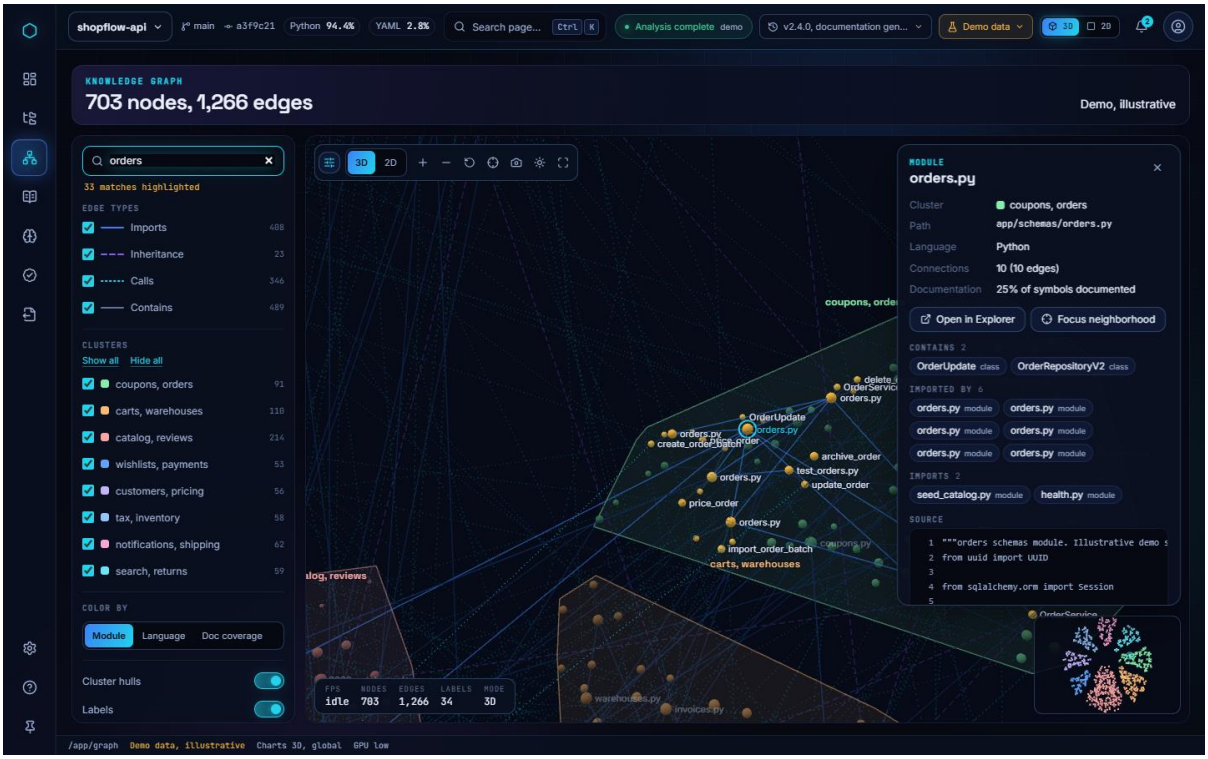


Fig 4.5 Knowledge Graph in 3D with a Selected Node (Demo Data, Illustrative)

4.1.6 Code Explorer and Verification Workspace

The Code Explorer (Fig 4.6) has three resizable panes; their sizes are saved per page and can be changed by mouse or keyboard. The file tree follows the WAI-ARIA tree pattern [43] and shows a documentation-coverage dot for each file. Code is highlighted with Shiki, using CSS variables mapped onto the design tokens, and the Shiki grammars are loaded lazily. The gutter shows D (documented) and U (undocumented) badges for each declaration. Hovering over a declaration for 280 ms opens a card with its signature and docstring. The inspector reads parameters, return statements and raised exceptions directly from the source lines, and every fact carries a source chip that scrolls to and flashes the line it came from.

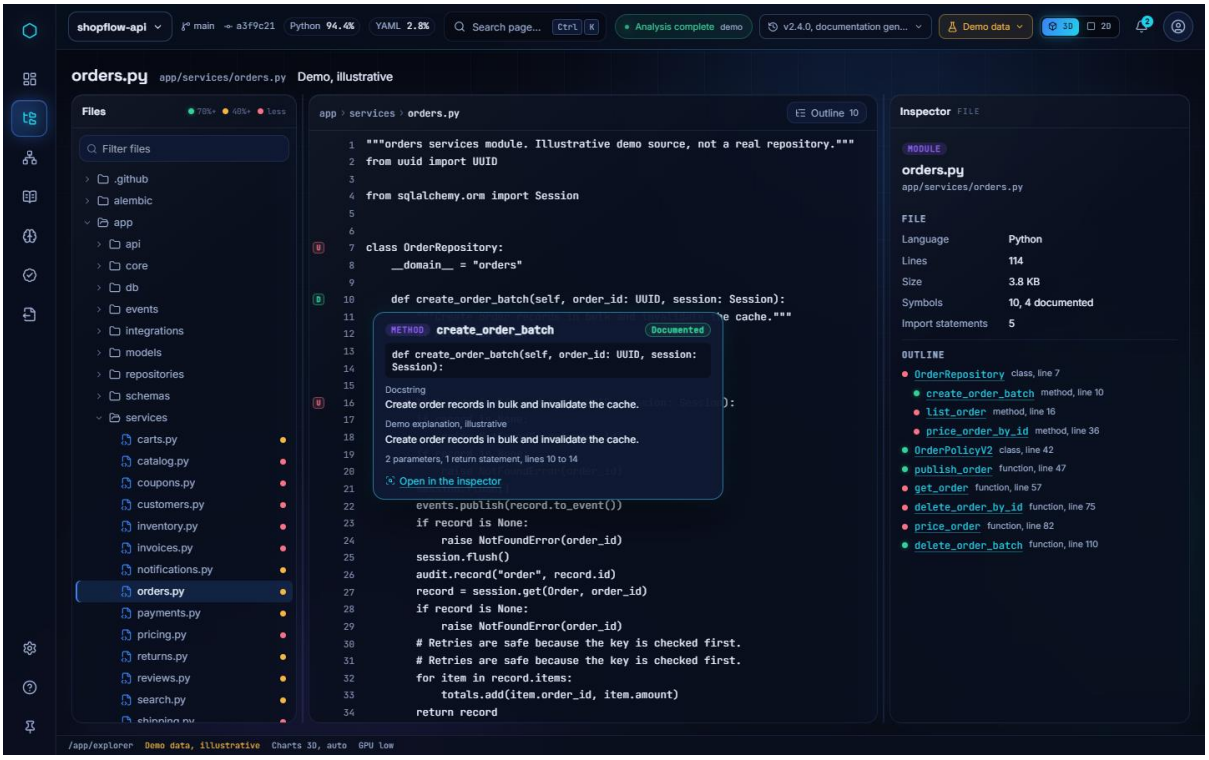


Fig 4.6 Code Explorer with Hover Card and Inspector (Demo Data, Illustrative)

The Verification workspace (Fig 4.7) shows the status totals as count buttons, a stacked bar and a score ring, followed by a windowed claim queue and an evidence view. For the selected claim, the evidence locator (Listing A.4) marks supporting lines in green and possibly contradicting lines in red, and an SVG connector links the claim card to the line. Each queue row shows "confidence n/a", because the claim record contains no confidence score and the interface does not invent one. Reviewer decisions are saved in the browser for each repository scope. In Live mode the page explains that claim data is pending integration.

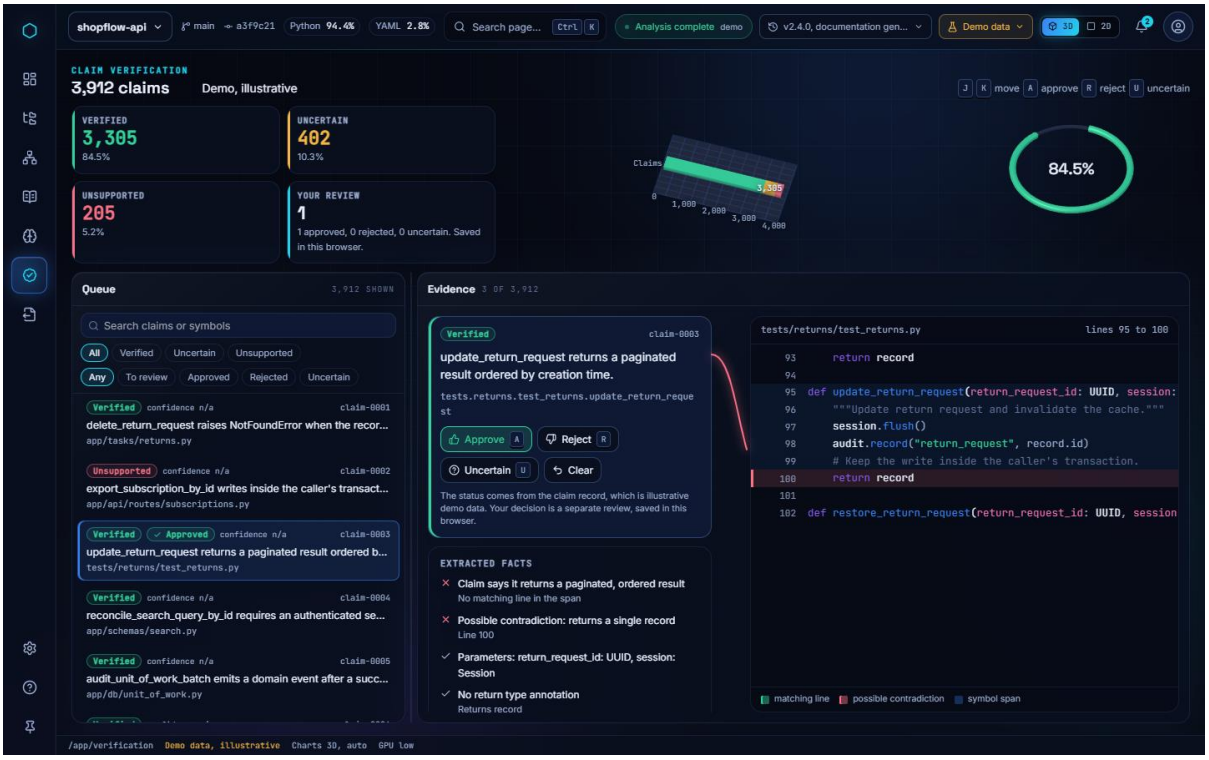


Fig 4.7 Verification Workspace with Evidence Lines (Demo Data, Illustrative)

4.2 TESTING

Testing combines backend unit, integration and property-based tests, frontend unit tests, browser tests with accessibility checks, and performance measurements on real hardware. In line with the reporting practice of the project, this section separates **observed results**, which were executed and recorded in the evidence folder, from **suggested procedures**, which are marked *(Not Executed)*.

4.2.1 Backend Testing (Observed)

The backend suite contains eleven test files with 53 test functions; parametrization raises the number of executed cases. Property-based tests use Hypothesis [46]. Table 4.3 lists the files and Table 4.4 the recorded runs.

Table 4.3 Backend Test Files

Test file	Test functions	Area covered
test_ingestion.py	11	File/ZIP validation, limits, traversal, symlinks, encodings
test_github.py	8	URL canonicalization, redirects, DNS/IP pinning, bounded download
test_preprocessing.py	8	Tokens, comments, docstrings, three views, spans

Test file	Test functions	Area covered
test_evaluation.py	7	Precision/recall/F1 scorer
test_api.py	6	Endpoints, API key, error envelopes
test_phase_two.py	4	Symbols, edges, resolution, statistics
test_pipeline.py	4	End-to-end preprocessing pipeline
test_jobs.py	2	Deterministic jobs and reuse
test_contracts.py, test_live_worker.py, test_postgres.py	1 each	Schema contracts, Celery worker, PostgreSQL

Table 4.4 Recorded Backend Test Results

Run	Result	Coverage	Evidence
Phase I, native and container (PostgreSQL)	69 passed, 0 skipped	95%	backend-postgres.txt, backend-container.txt
Phase I, fast suite	67 passed, 2 skipped	not recorded	Phase I verification log
Phase II, container (final)	73 passed	94%	phase2-backend-container.txt
Phase II remediation	80 passed, 0 skipped	94%	remediation-backend.txt
Runtime smoke test	Import 201; original bytes identical; 2 symbols; 1 relationship	n/a	runtime-smoke.json

4.2.2 Frontend Testing (Observed)

Frontend unit tests run with Vitest. Browser tests run with Playwright in Chromium and include axe-core [42] accessibility scans. Table 4.5 lists the unit test files, Fig 4.8 shows how the unit suite grew across milestones, and Table 4.6 gives the recorded browser runs.

Table 4.5 Frontend Unit Test Files (Vitest, 113 Passing at M4)

Test file	Tests	Area covered
viz/__tests__/integrity.test.ts	33	Chart data integrity for all kinds
data/__tests__/data-layer.test.tsx	25	Demo and Live adapters, error typing, no fallback
viz/__tests__/controls.test.tsx	16	Container controls, table view, export
app/__tests__/graph.test.ts	14	Graph model, search ranking, hull, label budget
app/__tests__/shell.test.tsx	9	Shell, navigation, palette
data/__tests__/analysis.test.ts	9	Mapping of ExploratoryStatistics
app/__tests__/derive.test.ts	7	Derivation of VizModels

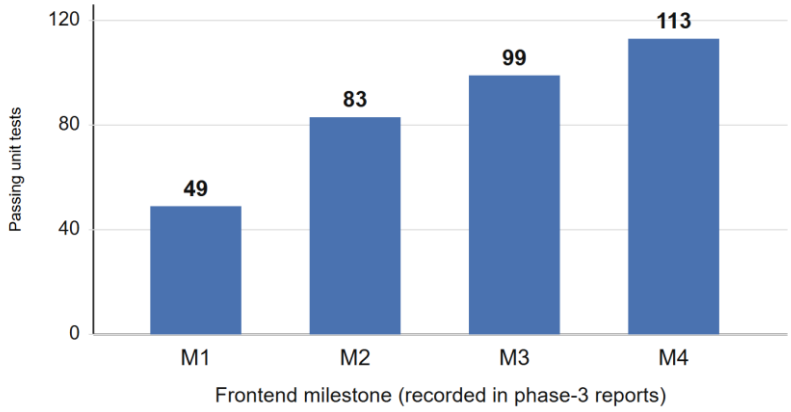


Fig 4.8 Passing Frontend Unit Tests by Milestone (Recorded)

Table 4.6 Recorded Browser Test Results (Playwright, Chromium)

Milestone	Suite	Real GPU	Software GL
M1	Design system and visualization	19 / 19	19 / 19
M2	Shell, exports and keyboard	73 / 73	72 / 73 (tooltip timing; rerun 5 / 5)
M3	Overview	6 / 6	Deferred to M7
M4	Knowledge Graph	11 / 11	Deferred to M7

Other recorded checks: `npm audit` reported 0 vulnerabilities, and axe-core reported 0 violations for the rule tags tested. Strict type checking and ESLint pass at each milestone gate.

4.2.3 Performance Testing (Observed)

Performance was measured in headless Chrome on the real GPU of the development laptop, an Intel HD 520 integrated graphics adapter. In the M1 chart showcase, the 3D network chart (132 nodes) improved from 42.6 frames per second at the M1 gate to 60.1 frames per second in all five runs after optimisation, with a p95 frame time of 16.8 ms and 1.5 ms of JavaScript per frame. All 22 chart stages share one WebGL context. Table 4.7 gives the Knowledge Graph benchmark.

Table 4.7 Knowledge Graph Frame Rate on Intel HD 520 (Recorded, Headless Chrome)

Mode	703 nodes	1,000 nodes	2,000 nodes
3D	60 fps (p95 16.8 ms)	60 fps (p95 16.8 ms)	60 fps (p95 16.7 ms)
2D, detail	30 fps (p95 99.7 ms)	30 fps (p95 133 ms)	15 fps (p95 83.3 ms)
2D, performance mode	60 fps (p95 33.3 ms)	60 fps (p95 33.7 ms)	30 fps (p95 50 ms)

The results show that the 3D renderer holds 60 frames per second up to 2,000 nodes on this hardware. The 2D renderer in detail mode does not: drawing the edges is its main cost on

integrated graphics, which is why the automatic performance mode exists. **No 60 frames per second claim is made for 2D detail mode, for software rendering, or for other devices**, because these were not measured.

4.2.4 Extraction Accuracy (Pending)

The scorer of Equations 3.4 and 3.5 is tested. The candidate lists (42 symbols, 50 relationships) are frozen, and the blank label forms are ready. No independent annotations have been completed, so the recorded result has the status REVIEW_PENDING and the metric values are null. Accuracy figures will be reported only after at least one independent annotator has labelled the corpus.

4.2.5 Suggested Test Procedures (Not Executed)

The following procedures are planned but have not been executed. No result is claimed for any of them.

Table 4.8 Planned Test Procedures

Procedure	Purpose	Status
Full Playwright suite on real GPU and software GL	Regression across all pages at the M7 gate	Not Executed
Unit and browser tests for Explorer and Verification	Cover tree keyboard navigation, inspector, queue keys, evidence locator	Not Executed
Live adapter against the Docker Compose stack	Confirm the dashboard with PostgreSQL and the Celery worker	Not Executed
Live GitHub import	Exercise the pinned network path against github.com	Not Executed
Remote CI workflow (verify.yml)	Independent reproduction of the test runs	Not Executed
Screen reader and physical mobile devices	Accessibility beyond automated axe rules	Not Executed
Gold-standard annotation and scoring	Precision/recall/F1 of extraction	Not Executed

4.2.6 Known Limitations

- Phase II has no gold annotations, so extraction accuracy is unknown.
- Java analysis has no type resolution, and dynamic dispatch, wildcard imports and transitive dependencies are not resolved. Parameters are not emitted as separate symbols.
- The blob store uses POSIX-only file operations, so the backend runs on Linux and in containers but not natively on Windows. A failed database transaction can leave an orphan blob.

- Deployment is single-tenant, without TLS termination or rate limiting.
- The visualization requirements VIS-01 to VIS-05 are deferred until explicitly approved. The dashboard covers them only partially.
- Docs Studio, Intelligence, Exports and the marketing site are not built. All NLP models (NLP-01 to NLP-05) and advanced features (ADV-01 to ADV-06) are deferred to Phases IV and V.
- Browser testing covers Chromium only.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

CodeAtlas AI set out to make statements about source code traceable to evidence. The work completed so far builds the foundation such a system requires.

- Safe, reproducible collection of Python and Java code, with byte-exact preservation.
- An NLP-ready preprocessing pipeline that produces three aligned text views.
- Structure extraction whose every relationship states how certain it is.
- A set of deterministic statistics that treat the repository as a text corpus.
- An interactive dashboard: a shared 3D/2D visualization system, an Overview page, a Knowledge Graph that runs at 60 frames per second in 3D on integrated graphics, and a Code Explorer and Verification workspace that link facts to source lines.

The project is reported honestly as **partial**. Phase I has passed its final verification audit. Phase II is implemented but remains REVIEW_PENDING until gold annotations exist. Phase III has completed milestones M1 to M4, has built the Explorer and Verification pages, and has not yet built Docs Studio, Intelligence, Exports, the marketing site or the final polish milestone. Phases IV and V have not started. Table 5.1 summarises the status.

Table 5.1 Final Status Summary

Area	Status	Evidence
Phase I: collection and preprocessing	PASS	69 tests passed, 95% coverage; final verification audit
Phase II: exploratory analysis and structure	PARTIAL / REVIEW_PENDING	80 tests passed, 94% coverage; gold labels = 0
Phase III M1–M4: visualization, shell, Overview, Graph	Complete	113 unit tests; browser suites as in Table 4.6; benchmarks
Phase III M5: Explorer, Verification	Built, tests pending	Manual browser checks
Phase III M5–M7: Docs, Intelligence, Exports, site, polish	Not started	None
Phase IV: NLP models	Not started	Placeholder folders only
Phase V: advanced implementation	Not started	None

5.2 FUTURE SCOPE

- **Gold-standard evaluation.** Collect independent annotations for the frozen candidates and report precision, recall and F1 for symbols and relationships. Then extend the corpus to real open-source repositories, split by project [11], [22].
- **Deterministic documentation baseline (NLP-01).** Generate template documentation from signatures, docstrings and extracted facts, as a reference against which models are measured.
- **Code NER and transformer summarization (NLP-02 to NLP-04).** Fine-tune and evaluate CodeT5 [17] for function and module summaries. Report BLEU and ROUGE-L [24], [25] together with human judgements, because of the known weaknesses of overlap metrics [23].
- **Claim verification.** Split generated documentation into atomic claims [28] and check each against the extracted structure and source spans, so that Verified, Uncertain and Unsupported come from evidence rather than from fixtures. Replace the rule-based locator with retrieval over symbol spans [27].
- **Remaining Phase III pages.** Docs Studio, Intelligence and Exports (Markdown ZIP, static site, PDF, JSON and CSV), the marketing site, and the M7 audit, including the full browser suite, software-GL runs and a screen reader pass.
- **Platform hardening.** Portable blob storage for Windows, removal of orphan blobs, multi-tenant access control, TLS and rate limiting, and a live run of the CI workflow.

APPENDIX A: SOURCE CODE

Selected excerpts from the repository are reproduced below without modification, apart from omitted lines marked "...".

Listing A.1 NLP view of a token (analyzers/preprocessing.py)

```
def _nlp(text: str, kind: str, config: PreprocessingConfig) -> str:
    if kind == "identifier" and config.split_identifiers:
        text = re.sub(r"([a-z0-9])([A-Z])", r"\1 \2", text).replace("_", " ")
    elif config.filter_comment_markers and kind in {"comment", "javadoc"}:
        text = re.sub(r"(?m)^\s*(?:#|//|/\*+|\*+/|\*)\s?", "", text)
        text = re.sub(r"\*/\s*$", "", text)
    return " ".join(text.split())
```

Listing A.2 Import cycle detection with Tarjan's algorithm (analyzers/exploration.py)

```
def _cycles(adjacency: dict[str, set[str]]) -> list[list[str]]:
    # Tarjan SCCs; each member is a real module in the snapshot.
    order = 0
    low: dict[str, int] = {}
    index: dict[str, int] = {}
    stack: list[str] = []
    active: set[str] = set()
    found: list[list[str]] = []

    def visit(node: str):
        nonlocal order
        index[node] = low[node] = order
        order += 1
        stack.append(node)
        active.add(node)
        for neighbor in sorted(adjacency.get(node, ())):
            if neighbor not in index:
                visit(neighbor)
                low[node] = min(low[node], low[neighbor])
            elif neighbor in active:
                low[node] = min(low[node], index[neighbor])
        if low[node] == index[node]:
            component = []
            while True:
                member = stack.pop()
                active.remove(member)
                component.append(member)
                if member == node:
                    break
            if len(component) > 1 or node in adjacency.get(node, ()):
                found.append(sorted(component))

    for node in sorted(adjacency):
        if node not in index:
            visit(node)
    return sorted(found)
```

Listing A.3 Relationship resolution and lexical diversity (analyzers/exploration.py)

```
        if edge.kind == "imports":
            ...
            candidates = [s for name, s in module_symbols.items() if name ==
target_name]
            if len(candidates) == 1:
                target, status = candidates[0], "confirmed"
                graph[source_name].add(target.qualified_name)
                fanout[source_name] += 1
                fanin[target.qualified_name] += 1
            elif len(candidates) > 1:
                status = "ambiguous"
            else:
                status = "external" if not edge.target_name.startswith(".") else
"unresolved"
        ...
        elif edge.kind == "calls" and status != "confirmed":
            # A bare call is resolved only when exactly one local callable has that
name.
            candidates = [s for s in symbols if s.name == edge.target_name
                          and s.kind in {"function", "method", "constructor"}
                          and s.file_id == edge.evidence.file_id]
            if len(candidates) == 1 and "." not in edge.target_name:
                target, status = candidates[0], "confirmed"
            elif len(candidates) > 1:
                status = "ambiguous"
    ...
        vocabulary_size=len(set(words)),
        lexical_diversity=len(set(words)) / len(words) if words else 0,
```

Listing A.4 Evidence locator (frontend/src/app/pages/verification/evidence.ts)

```
/** Rule-based line matching for claims. ... It is not the verifier, it does not
 *  decide a claim's status, and a match is not proof that the claim is true. */
export function ruleFor(claim: string): EvidenceRule | null {
  const raised = claim.match(/\braises\s+([A-Za-z_][\w.]*)/);
  if (raised) return { id: 'raises', assertion: `raises ${raised[1]}`,
    support: new RegExp(`\\b(raise|throw)\\s+(new\\s+)?${escape(raised[1])}\\b`) };
  if (/emits? (a )?(domain )?event/i.test(claim)) return { id: 'event', ... };
  if (/caller'?s transaction/i.test(claim)) return { id: 'transaction', ...,
    contradict: /\.commit\(\)/, contradictReason: 'commits its own transaction' };
  ...
  return null;
}

export function locateEvidence(claim: string, lines: string[], start: number, end:
number): Evidence {
  const rule = ruleFor(claim);
  const support: number[] = [], contradict: number[] = [];
  if (rule) for (let line = start; line <= Math.min(end, lines.length); line += 1) {
    const text = lines[line - 1];
    if (rule.support.test(text)) support.push(line);
    else if (rule.contradict?.test(text)) contradict.push(line);
  }
  return { rule, support, contradict };
}
```

Listing A.5 Data-layer interface and graph helpers (frontend/src/data/dashboard.ts, graph/model.ts)

```
export interface DashboardApi {
  readonly mode: DataMode;
  getHealth(): Promise<ServiceHealth>;
  getRepositories(): Promise<Repository[]>;
  getSnapshots(repositoryId: string): Promise<SnapshotInfo[]>;
  getAnalysisStatus(scope: RepositoryScope): Promise<AnalysisStatus>;
  getOverview(scope?: RepositoryScope): Promise<Overview>;
  getFileTree(scope?: RepositoryScope): Promise<FileNode[]>;
  getFile(path: string, scope?: RepositoryScope): Promise<FileDocument>;
  getGraph(scope?: RepositoryScope): Promise<DashboardGraph>;
  getDocs(scope?: RepositoryScope): Promise<DocsReport>;
  ...
}

// frontend/src/app/pages/graph/model.ts
export const nodeRadius = (node: GNode) => 2.6 + Math.sqrt(node.degree) * 1.7;
export function labelBudget(zoom: number, count: number): number {
  if (zoom < .45) return Math.min(count, 14);
  if (zoom < .8) return Math.min(count, 24);
  if (zoom < 1.4) return Math.min(count, 70);
  if (zoom < 2.2) return Math.min(count, 180);
  return count;
}
```

APPENDIX B: SCREENSHOTS

Screenshots were captured from the running application in Chromium during milestone verification. Pages showing the *shopflow-api* repository use the Demo data layer, and their values are illustrative.

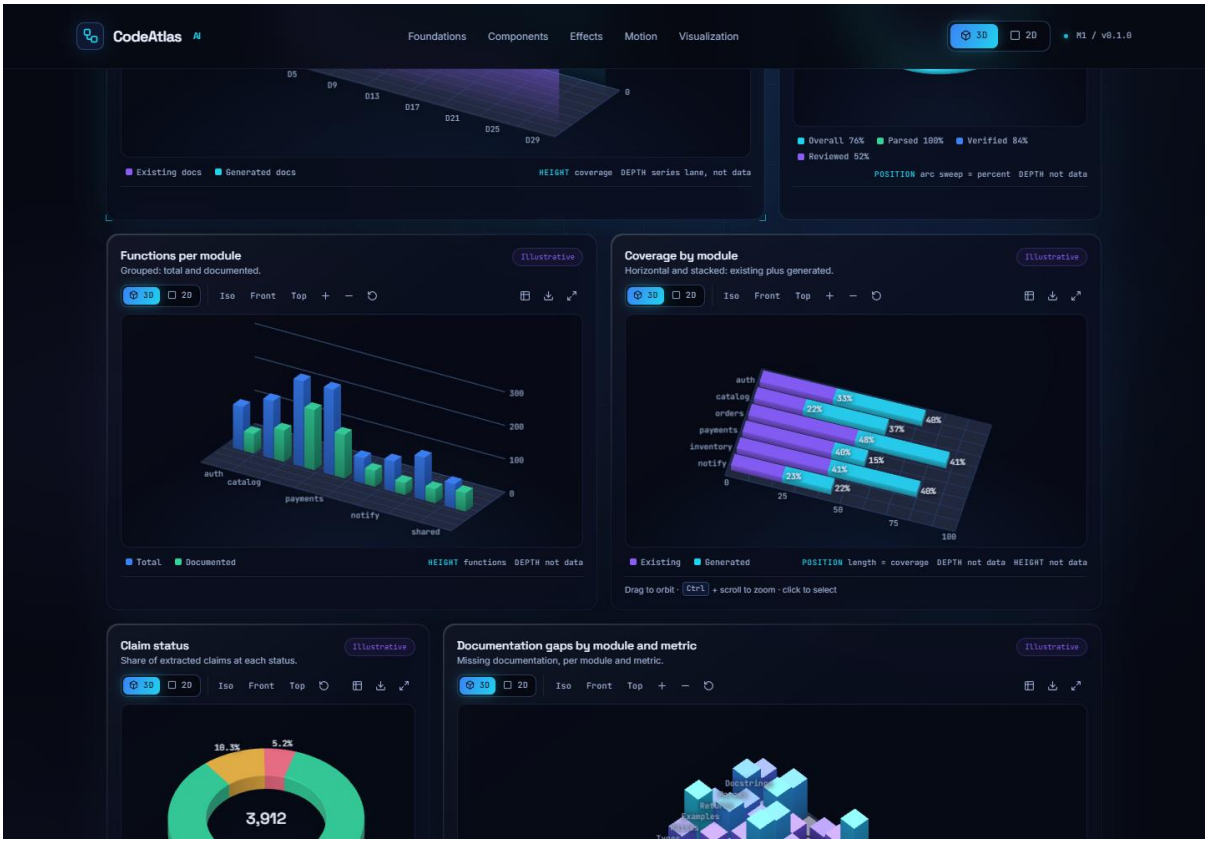


Fig B.1 Design-system showcase: grouped and stacked bars, 3D



Fig B.2 Design-system showcase: grouped and stacked bars, 2D

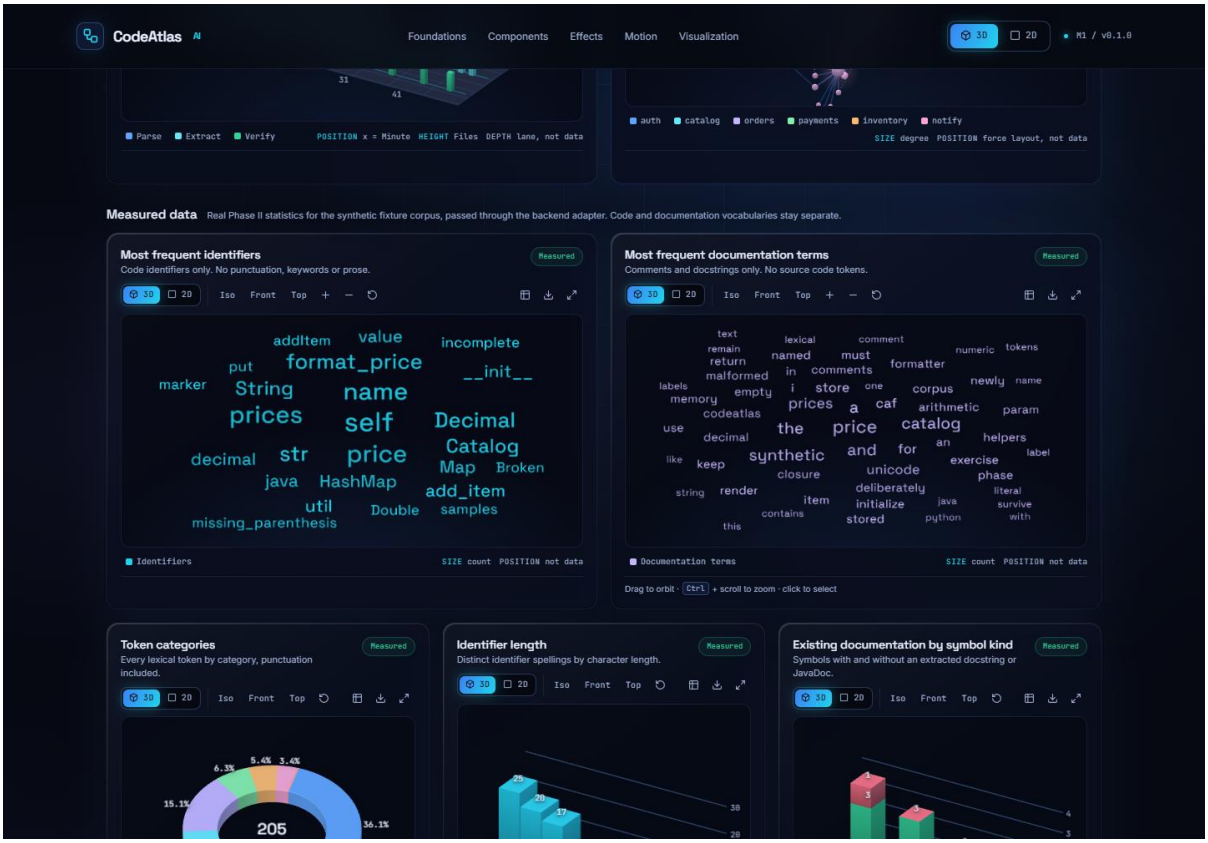


Fig B.3 Measured word clouds of identifiers and documentation terms from the Phase II analysis, 3D

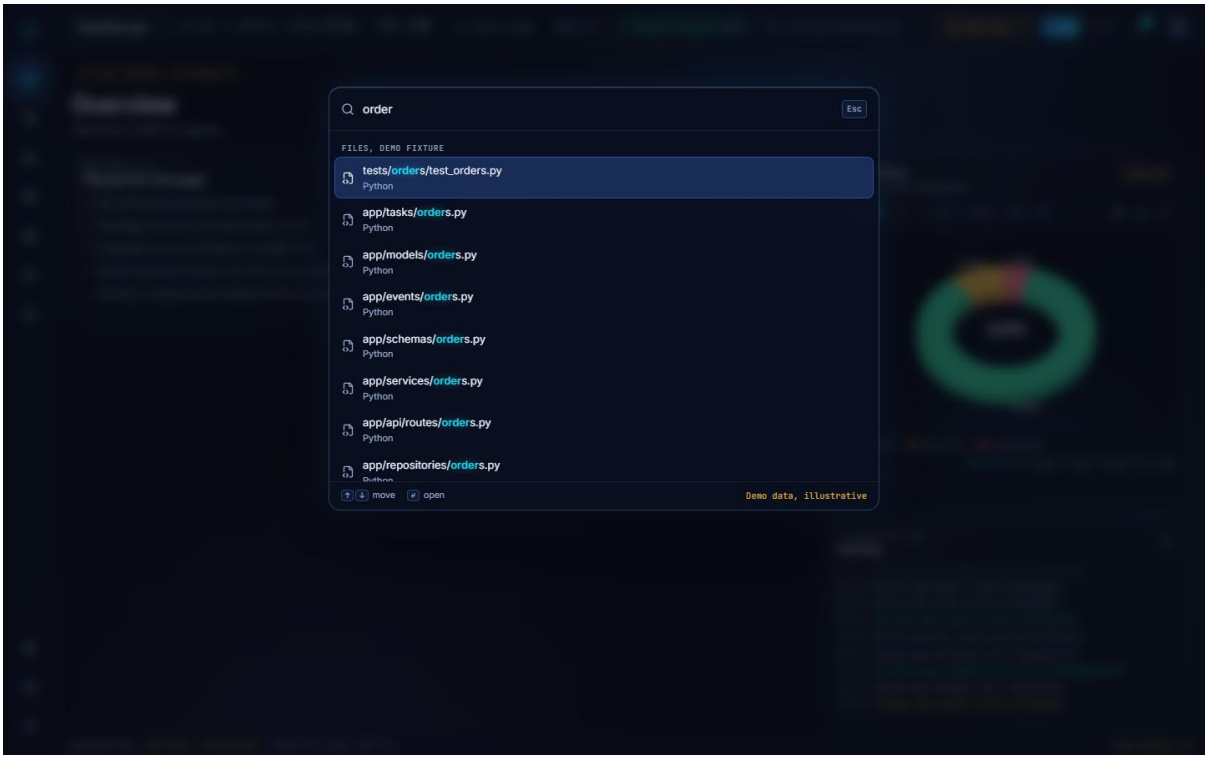


Fig B.4 Command palette in the dashboard shell

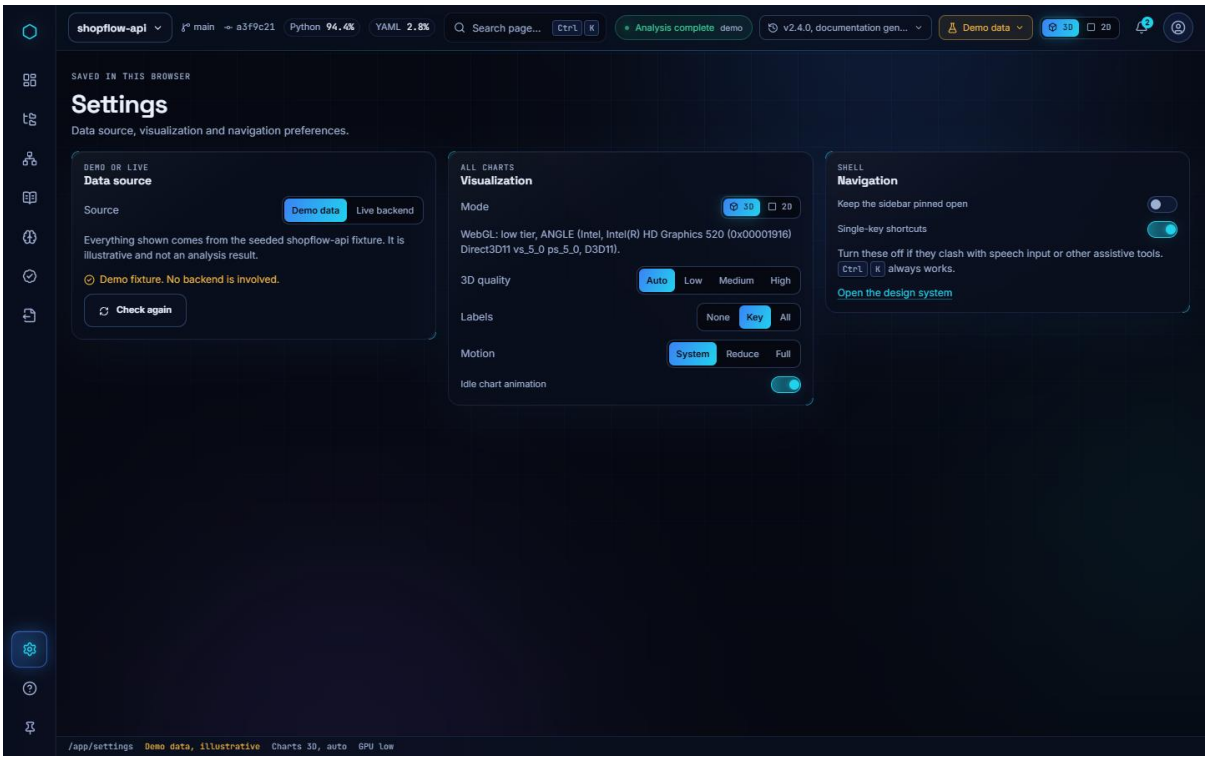


Fig B.5 Settings page with data mode and Live connection

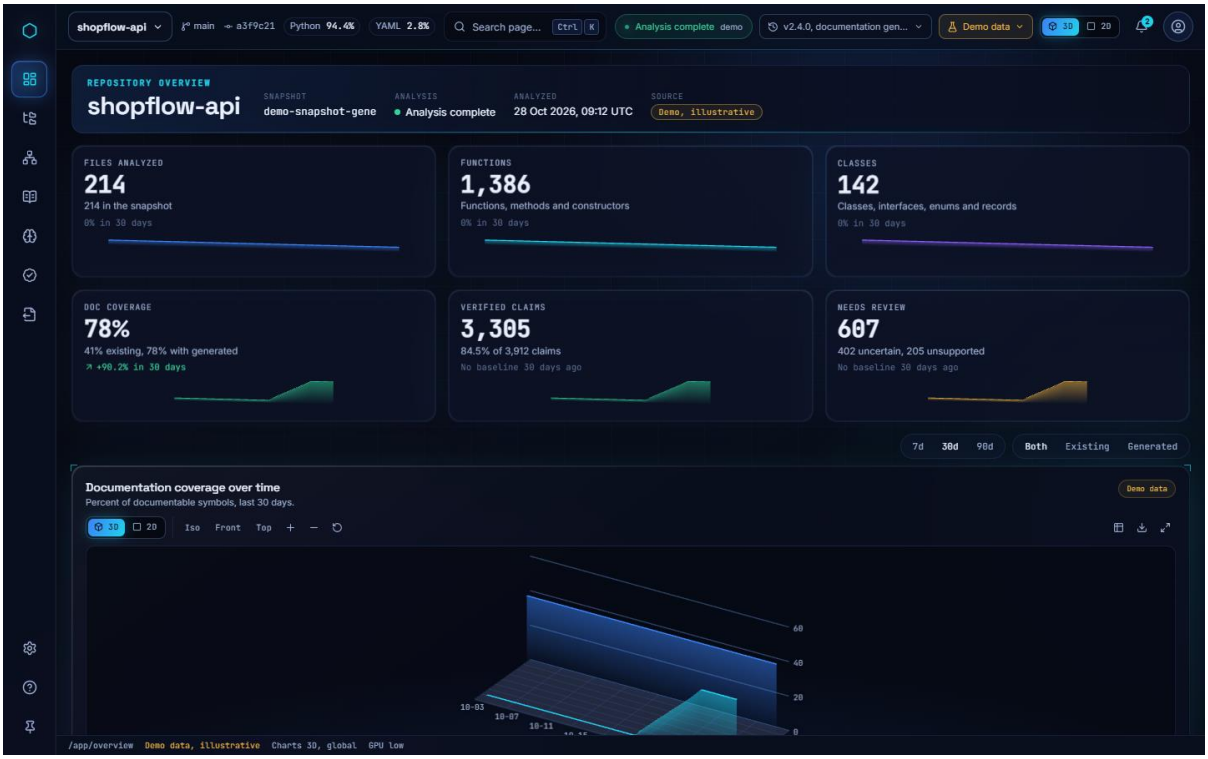


Fig B.6 Overview page in 3D mode

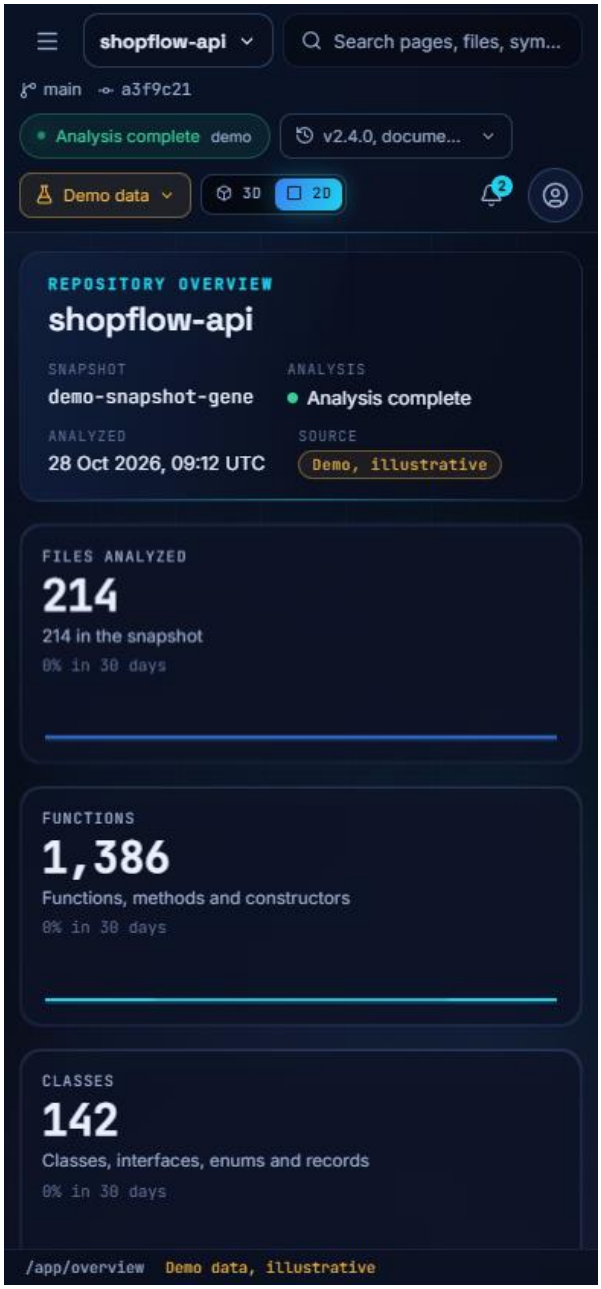


Fig B.7 Overview page at phone width (390 px)

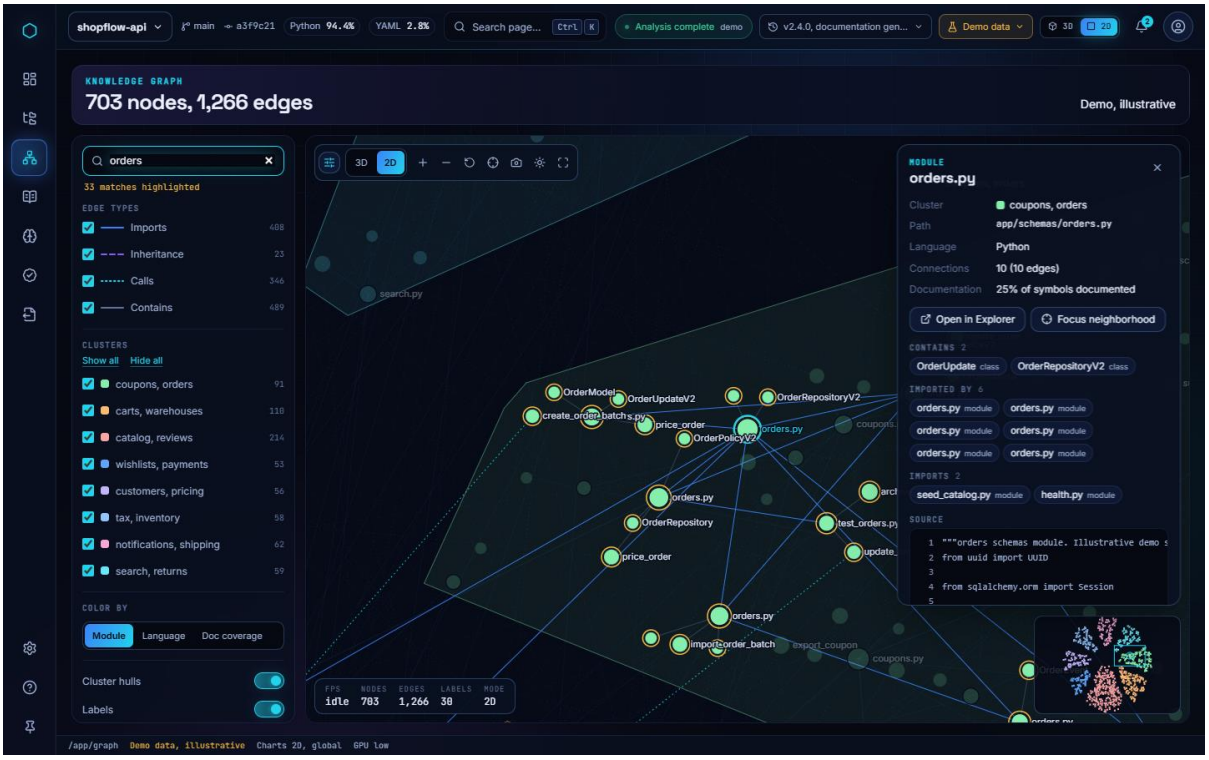


Fig B.8 Knowledge Graph in 2D with a selected node

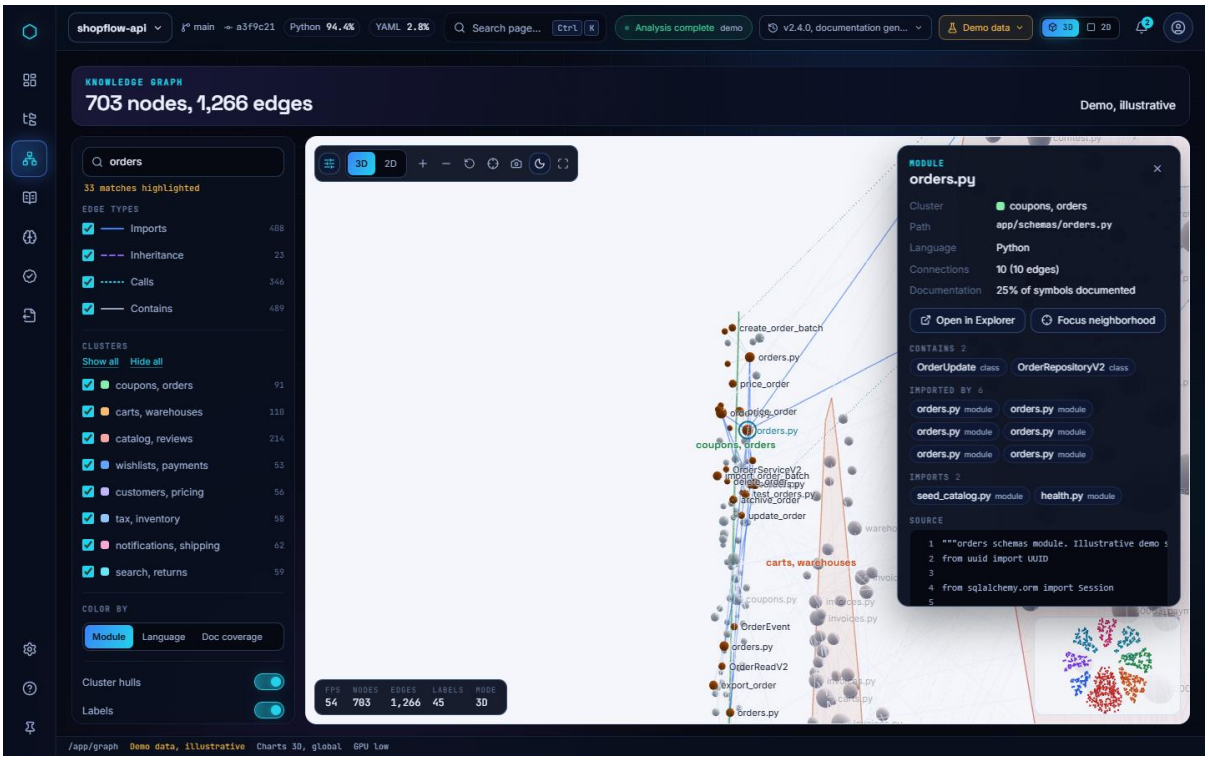


Fig B.9 Knowledge Graph in the light theme

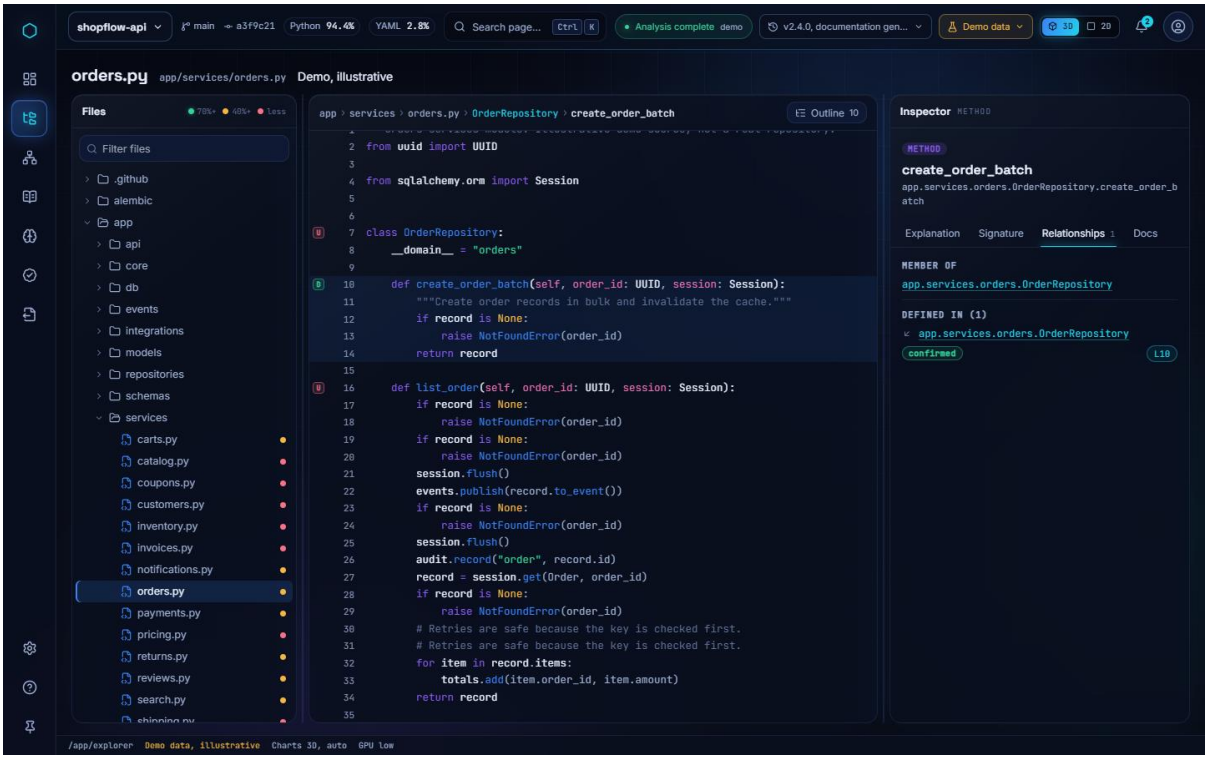


Fig B.10 Code Explorer, Relationships tab of the inspector

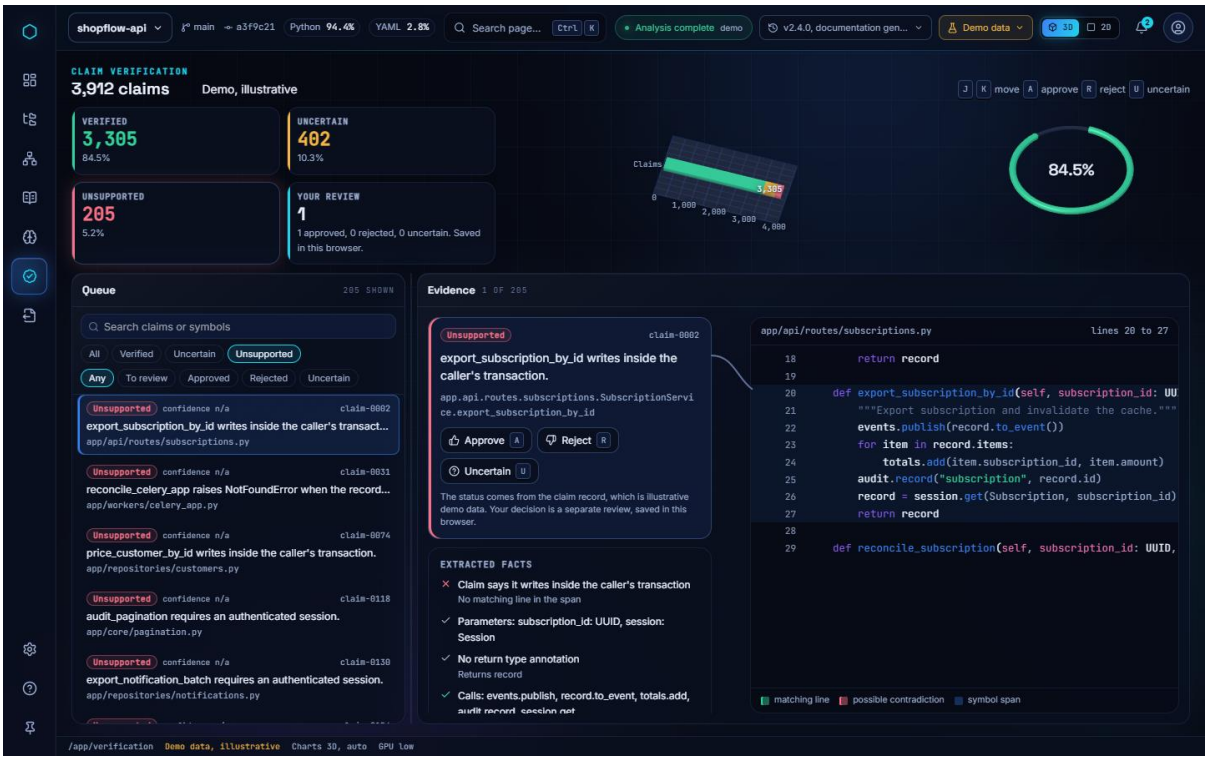


Fig B.11 Verification workspace filtered to unsupported claims

REFERENCES

[1] Oracle, "Javadoc tool," *Java SE Documentation*. [Online]. Available: https://docs.oracle.com/en/java/javase/

[2] D. van Heesch, "Doxygen: Generate documentation from source code." [Online]. Available: https://www.doxygen.nl

[3] G. Brandl and the Sphinx team, "Sphinx: Python documentation generator." [Online]. Available: https://www.sphinx-doc.org

[4] G. Sridhara, E. Hill, D. Muppaneni, L. Pollock, and K. Vijay-Shanker, "Towards automatically generating summary comments for Java methods," in *Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2010, pp. 43–52.

[5] S. Haiduc, J. Aponte, L. Moreno, and A. Marcus, "On the use of automated text summarization techniques for summarizing source code," in *Proc. 17th Working Conf. Reverse Engineering (WCRE)*, 2010, pp. 35–44.

[6] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, "Summarizing source code using a neural attention model," in *Proc. 54th Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2016, pp. 2073–2083.

[7] X. Hu, G. Li, X. Xia, D. Lo, and Z. Jin, "Deep code comment generation," in *Proc. 26th Int. Conf. Program Comprehension (ICPC)*, 2018, pp. 200–210.

[8] U. Alon, S. Brody, O. Levy, and E. Yahav, "code2seq: Generating sequences from structured representations of code," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[9] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "code2vec: Learning distributed representations of code," *Proc. ACM Program. Lang.*, vol. 3, no. POPL, pp. 40:1–40:29, 2019.

[10] A. LeClair, S. Jiang, and C. McMillan, "A neural model for generating natural language summaries of program subroutines," in *Proc. 41st Int. Conf. Software Engineering (ICSE)*, 2019, pp. 795–806.

[11] A. LeClair and C. McMillan, "Recommendations for datasets for source code summarization," in *Proc. NAACL-HLT*, 2019, pp. 3931–3937.

[12] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.

[13] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[14] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "A transformer-based approach for source code summarization," in *Proc. 58th Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2020, pp. 4998–5007.

[15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[16] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Assoc. Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[17] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 8696–8708.

[18] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[19] S. Lu *et al.*, "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.

[20] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Computing Surveys*, vol. 51, no. 4, pp. 81:1–81:37, 2018.

[21] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu, "On the naturalness of software," in *Proc. 34th Int. Conf. Software Engineering (ICSE)*, 2012, pp. 837–847.

[22] M. Allamanis, "The adverse effects of code duplication in machine learning models of code," in *Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, pp. 143–153.

[23] D. Roy, S. Fakhoury, and V. Arnaoudova, "Reassessing automatic evaluation metrics for code summarization tasks," in *Proc. 29th ACM Joint European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, 2021, pp. 1105–1116.

[24] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2002, pp. 311–318.

[25] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out: Proc. ACL-04 Workshop*, 2004, pp. 74–81.

[26] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 248:1–248:38, 2023.

[27] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[28] S. Min *et al.*, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 12076–12100.

[29] M. Brunsfeld *et al.*, "Tree-sitter: An incremental parsing system for programming tools." [Online]. Available: https://tree-sitter.github.io/tree-sitter/

[30] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Computing*, vol. 1, no. 2, pp. 146–160, 1972.

[31] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Information Processing Letters*, vol. 9, no. 5, pp. 216–219, 1979.

[32] T. M. J. Fruchterman and E. M. Reingold, "Graph drawing by force-directed placement," *Software: Practice and Experience*, vol. 21, no. 11, pp. 1129–1164, 1991.

[33] M. Bostock, V. Ogievetsky, and J. Heer, "D3: Data-driven documents," *IEEE Trans. Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011.

[34] B. Shneiderman, "The eyes have it: A task by data type taxonomy for information visualizations," in *Proc. IEEE Symp. Visual Languages*, 1996, pp. 336–343.

[35] M. C. Templin, *Certain Language Skills in Children: Their Development and Interrelationships*. Minneapolis, MN, USA: Univ. of Minnesota Press, 1957.

[36] E. Aghajani, C. Nagy, O. L. Vega-Márquez, M. Linares-Vásquez, L. Moreno, G. Bavota, and M. Lanza, "Software documentation issues unveiled," in *Proc. 41st Int. Conf. Software Engineering (ICSE)*, 2019, pp. 1199–1210.

[37] T. C. Lethbridge, J. Singer, and A. Forward, "How software engineers use documentation: The state of the practice," *IEEE Software*, vol. 20, no. 6, pp. 35–39, 2003.

[38] S. Ramírez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com

[39] The Celery Project, "Celery: Distributed task queue." [Online]. Available: https://docs.celeryq.dev

[40] GitHub, "REST API endpoints for repository contents." [Online]. Available: https://docs.github.com/en/rest/repos/contents

[41] three.js authors, "three.js: JavaScript 3D library." [Online]. Available: https://threejs.org

[42] Deque Systems, "axe-core: Accessibility engine for automated web UI testing." [Online]. Available: https://github.com/dequelabs/axe-core

[43] W3C Web Accessibility Initiative, "ARIA Authoring Practices Guide: Tree view pattern." [Online]. Available: https://www.w3.org/WAI/ARIA/apg/patterns/treeview/

[44] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[45] Python Software Foundation, "ast — Abstract syntax trees," *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/ast.html

[46] D. R. MacIver, Z. Hatfield-Dodds *et al.*, "Hypothesis: A new approach to property-based testing," *J. Open Source Software*, vol. 4, no. 43, p. 1891, 2019.